



BÁO CÁO THỰC HÀNH

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

Môn học: Công nghệ DevOps và ứng dụng

Lớp: NT548.Q21

THÀNH VIÊN THỰC HIỆN (Nhóm 03):

STT	Họ và tên	MSSV
1	Dương Thanh Huyền	23520659
2	Đoàn Thanh Thảo	23521466
3	Nguyễn Cao Thông	23521524
4	Nguyễn Lê Như Thuận	23521551
5	Võ Trần Việt Tiến	23521590

Điểm tự đánh giá
10

ĐÁNH GIÁ KHÁC:

Tổng thời gian thực hiện	
Phân chia công việc	
Ý kiến (nếu có) + Khó khăn + Đề xuất, kiến nghị	

Phần bên dưới của báo cáo này là báo cáo chi tiết của nhóm thực hiện

MỤC LỤC

BÁO CÁO CHI TIẾT	3
Câu 1 — Triển khai hạ tầng AWS bằng Terraform	3
1.1. Mục tiêu và kiến trúc tổng quan.....	3
1.2. Cấu trúc thư mục Terraform.....	3
1.3. Triển khai từng module	4
1.4. Quy trình triển khai	10
1.5. Xác minh kết quả trên giao diện AWS (AWS Console).....	13
1.6. Test cases và xác minh	18
Câu 2 — Triển khai hạ tầng AWS bằng CloudFormation	21
2.1. Mục tiêu và kiến trúc	21
2.2. Cấu trúc thư mục CloudFormation.....	22
2.3. Triển khai các template	22
2.4. Quy trình triển khai	27
2.5. Xác minh kết quả trên giao diện AWS (AWS Console)	28
2.6. Test cases và xác minh	35
Câu 3 — Tổng kết và so sánh.....	38
3.1. So sánh Terraform và CloudFormation.....	38
3.2. Bài học kinh nghiệm.....	38
3.3. Kết luận.....	39
PHỤ LỤC	40
A. Hướng dẫn chạy mã nguồn.....	40
B. Tham khảo	40

BÁO CÁO CHI TIẾT

Câu 1 — Triển khai hạ tầng AWS bằng Terraform

1.1.Mục tiêu và kiến trúc tổng quan

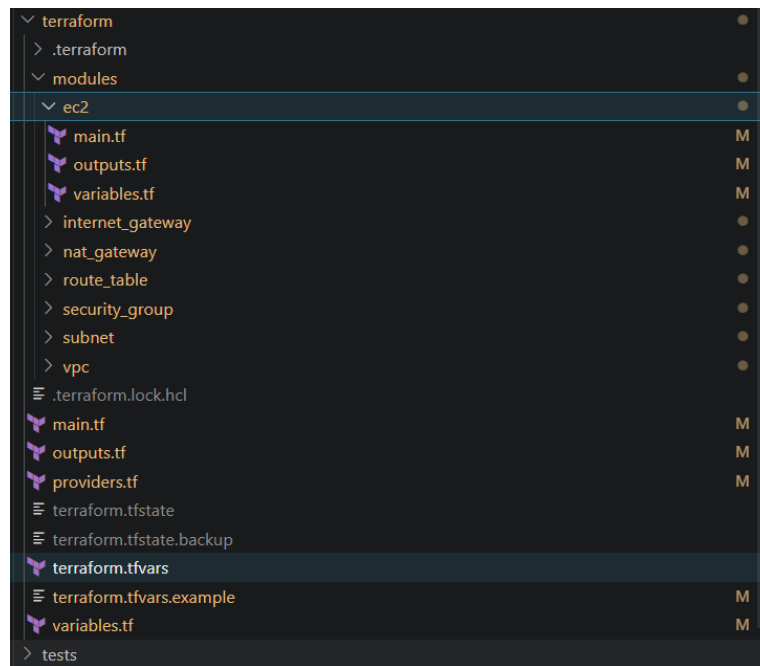
Mục tiêu: Sử dụng Infrastructure as Code (IaC) với công cụ **Terraform** để tự động hóa việc cấp phát và quản lý hạ tầng mạng cơ bản trên AWS. Các tài nguyên được triển khai bao gồm: VPC, Subnets (Public & Private), Internet Gateway, NAT Gateway, Route Tables, EC2 instances và Security Groups.

Bảng các resource sẽ tạo:

Resource	Số lượng	CIDR / Thông tin	Chức năng chính
VPC	1	10.0.0.0/16	Mạng ảo cô lập chứa toàn bộ hạ tầng lab.
Public Subnet	1	10.0.1.0/24	Chứa tài nguyên có thể truy cập từ Internet (Public EC2, NAT GW).
Private Subnet	1	10.0.2.0/24	Chứa tài nguyên nội bộ, không có Public IP (Private EC2).
Internet Gateway	1	-	Cổng giao tiếp giữa VPC và Internet.
NAT Gateway	1	Kèm 1 Elastic IP	Cho phép Private EC2 đi ra Internet để cập nhật gói tin một chiều.
Route Tables	2	Public RT, Private RT	Bảng định tuyến điều hướng traffic cho các Subnet tương ứng.
Security Groups	3	Default, Public SG, Private SG	Tường lửa ảo kiểm soát luồng traffic Inbound/Outbound.
EC2 Instances	2	t2.micro (Amazon Linux 2)	Máy chủ ảo đóng vai trò tính toán trong các Subnet.

1.2.Cấu trúc thư mục Terraform

Dự án được xây dựng theo kiến trúc Module hóa (Modular Architecture), giúp mã nguồn có tính tái sử dụng cao, dễ quản lý và bảo trì.



1.3. Triển khai từng module

Kiến trúc mã nguồn được chia thành các module độc lập. Để minh họa cho nguyên tắc thiết kế chung:

- Module VPC (phần nền móng) sẽ được phân tích toàn diện cả 3 thành phần: định nghĩa tài nguyên (main.tf), biến đầu vào (variables.tf) và giá trị xuất (outputs.tf).
- Các module tiếp theo sau đó sẽ được thiết kế tương tự, do đó báo cáo sẽ chỉ tập trung phân tích logic cốt lõi tại main.tf.

1.3.7. Module VPC

✚ Mã nguồn main.tf:

```
lab01 > terraform > modules > vpc > main.tf
1  resource "aws_vpc" "main" {
2      cidr_block          = var.vpc_cidr
3      enable_dns_support  = true
4      enable_dns_hostnames = true
5
6      tags = {
7          Name = "${var.name_prefix}-vpc"
8      }
9  }
10
```

✚ Mã nguồn outputs.tf

```
lab01 > terraform > modules > vpc > outputs.tf
1  output "vpc_id" {
2  |      description = "The ID of the VPC"
3  |      value       = aws_vpc.main.id
4  |  }
5
6  output "vpc_cidr" {
7  |      description = "The CIDR block of the VPC"
8  |      value       = aws_vpc.main.cidr_block
9  |  }
10
```

🔗 Mã nguồn [variables.tf](#)

```
lab01 > terraform > modules > vpc > variables.tf
1  variable "name_prefix" {
2  |      description = "Prefix for resource names"
3  |      type        = string
4  |  }
5
6  variable "vpc_cidr" {
7  |      description = "CIDR block for the VPC"
8  |      type        = string
9  |  }
```

Phân tích kỹ thuật:

- **Thiết kế khối chức năng:** Module VPC được xây dựng như một hộp đen. Nó nhận đầu vào từ `variables.tf` (như dải mạng `vpc_cidr`) để khởi tạo linh hoạt mà không cần hard-code. Sau khi AWS cấp phát xong, `outputs.tf` sẽ làm nhiệm vụ "nhả" ra định danh `vpc_id` để làm tiền đề cho các module khác (Subnet, IGW) có thể gắn kết vào mạng lưới này.
- **Cấu hình mạng:** VPC khởi tạo với tính năng **`enable_dns_support`** và **`enable_dns_hostnames`** được bật (true). Điều này bắt buộc phải có để các tài nguyên bên trong (như EC2) tự động được AWS gán một DNS name nội bộ, giúp các dịch vụ dễ dàng phân giải tên miền.

1.3.2. Module Subnet

Lưu ý: Các file `variables.tf` và `outputs.tf` được thiết kế theo nguyên tắc tương tự module VPC. Dưới đây tập trung phân tích logic hạ tầng tại `main.tf`

🔗 Mã nguồn [main.tf](#):

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

```
lab01 > terraform > modules > subnet > main.tf
1  resource "aws_subnet" "public" {
2      vpc_id          = var.vpc_id
3      cidr_block       = var.public_subnet_cidr
4      availability_zone = var.availability_zone
5      map_public_ip_on_launch = true
6
7      tags = {
8          Name = "${var.name_prefix}-public-subnet"
9          Type = "Public"
10     }
11 }
12
13 resource "aws_subnet" "private" {
14     vpc_id          = var.vpc_id
15     cidr_block       = var.private_subnet_cidr
16     availability_zone = var.availability_zone
17     map_public_ip_on_launch = false
18
19     tags = {
20         Name = "${var.name_prefix}-private-subnet"
21         Type = "Private"
22     }
23 }
```

Phân tích kỹ thuật: Module này kế thừa `vpc_id` từ bước 1 để cắt không gian mạng lớn thành 2 mạng con:

- **Public Subnet:** Điểm khác biệt mấu chốt là cờ `map_public_ip_on_launch = true`. Thuộc tính này đảm bảo bất kỳ máy ảo (EC2) hay tài nguyên nào khởi tạo tại vùng này sẽ tự động nhận một IP Public, sẵn sàng cho việc giao tiếp trực tiếp với Internet.
- **Private Subnet:** Cờ này được đặt là `false`. Mạng này bị cô lập, ẩn hoàn toàn trước các truy cập trực tiếp từ bên ngoài để bảo vệ các máy chủ Backend hoặc Database.

1.3.3. Module Internet Gateway

🔗 Mã nguồn main.tf:

```
lab01 > terraform > modules > internet_gateway > main.tf
1  resource "aws_internet_gateway" "main" {
2      vpc_id = var.vpc_id
3
4      tags = {
5          Name = "${var.name_prefix}-igw"
6      }
7  }
```

Phân tích kỹ thuật: Internet Gateway (IGW) đóng vai trò là cửa ngõ logic hai chiều giữa hệ thống VPC nội bộ và môi trường Internet. Trong mã nguồn, IGW được attach trực tiếp vào hệ sinh thái thông qua tham số `vpc_id`. Bước này được thực hiện sớm để chuẩn bị nền tảng phần cứng ảo, làm cơ sở cho Bảng định tuyến (Route Table) sẽ điều hướng luồng giao thông mạng ở các bước sau.

1.3.4. Module NAT Gateway

🔗 Mã nguồn main.tf:

```
lab01 > terraform > modules > nat_gateway > main.tf
1 | # Elastic IP for NAT Gateway
2 | resource "aws_eip" "nat" {
3 |     domain = "vpc"
4 |
5 |     tags = {
6 |         Name = "${var.name_prefix}-nat-eip"
7 |     }
8 | }
9 |
10 | resource "aws_nat_gateway" "main" {
11 |     allocation_id = aws_eip.nat.id
12 |     subnet_id     = var.public_subnet_id
13 |
14 |     tags = {
15 |         Name = "${var.name_prefix}-nat-gw"
16 |     }
17 |
18 |     # NAT Gateway requires the EIP to be fully created first
19 |     depends_on = [aws_eip.nat]
20 | }
```

Phân tích kỹ thuật: Khác với IGW, NAT Gateway được sử dụng để cung cấp kết nối Internet một chiều cho các tài nguyên nằm trong Private Subnet (giúp các máy chủ tải bản cập nhật, cài đặt packages) mà vẫn ngăn chặn tuyệt đối các luồng truy cập khởi tạo từ bên ngoài. Cấu trúc module này thể hiện hai quyết định thiết kế quan trọng:

- **Cấp phát IP tĩnh (Elastic IP):** NAT Gateway trên AWS yêu cầu một địa chỉ IPv4 công cộng cố định để hoạt động. Khối `aws_eip` đảm nhận việc rút một IP từ Pool của AWS gán riêng cho NAT.
- **Quản lý vòng đời (Lifecycle) chặt chẽ:** Thuộc tính `depends_on = [aws_eip.nat]` là một Best Practice quan trọng trong Terraform. Do kiến trúc tạo tài nguyên bất đồng bộ của AWS, nếu Terraform cố gắng tạo NAT Gateway khi EIP chưa hoàn tất cấp phát, tiến trình sẽ báo lỗi (Race Condition). Khối lệnh này ép Terraform phải chờ EIP tạo xong mới tiến hành tạo NAT. Đồng thời, NAT được đặt chính xác vào `var.public_subnet_id` để nó có thể giao tiếp với Internet Gateway.

1.3.5. Module Route Table

🔗 Mã nguồn main.tf:

```
lab01 > terraform > modules > route_table > main.tf
21 | resource "aws_route_table" "private" {
22 |
23 |     route {
24 |         cidr_block     = "0.0.0.0/0"
25 |         nat_gateway_id = var.nat_gateway_id
26 |     }
27 |
28 |     tags = {
29 |         Name = "${var.name_prefix}-private-rt"
30 |     }
31 | }
32 |
33 |
34 | resource "aws_route_table_association" "private" {
35 |     subnet_id     = var.private_subnet_id
36 |     route_table_id = aws_route_table.private.id
37 | }
```

Phân tích kỹ thuật: Mạng Subnet và các Gateway đã có, nhưng để các gói tin biết đường di chuyển, hệ thống cần cấu hình Bảng định tuyến. Module này chia tách luồng traffic thành hai quy tắc (rules) hoàn toàn biệt lập:

- **Public Route:** Định nghĩa quy tắc cho mọi luồng dữ liệu đi ra Internet (0.0.0.0/0) trở thẳng ra `gateway_id = var.internet_gateway_id` (IGW). Khởi tạo `aws_route_table_association` sau đó ánh xạ chặt bảng này vào Public Subnet.
- **Private Route:** Cùng là hướng ra Internet (0.0.0.0/0), nhưng đích đến (Target) lại trở vào `nat_gateway_id = var.nat_gateway_id` (NAT Gateway). Nhờ thiết kế phân tách này, hạ tầng mạng được định tuyến luồng dữ liệu (traffic flow) một cách nghiêm ngặt, tuân thủ đúng kiến trúc bảo mật chuẩn trên môi trường Cloud.

1.3.6. Module Security Group

 Mã nguồn `main.tf`:

```
lab01 > terraform > modules > security_group > main.tf
1  # --- Default VPC Security Group ---
2  resource "aws_default_security_group" "default" {
3    vpc_id = var.vpc_id
4
5    tags = {
6      Name = "${var.name_prefix}-default-sg"
7    }
8  }
9
10 # --- Public EC2 Security Group ---
11 resource "aws_security_group" "public_ec2" {
12   name        = "${var.name_prefix}-public-ec2-sg"
13   description = "Allow SSH from my IP only"
14   vpc_id      = var.vpc_id
15
16   # SSH from specific IP only
17   ingress {
18     description = "SSH from my IP"
19     from_port   = 22
20     to_port     = 22
21     protocol    = "tcp"
22     cidr_blocks = [var.my_ip]
23   }
24
25   # Allow all outbound traffic
26   egress {
27     description = "Allow all outbound"
28     from_port   = 0
29     to_port     = 0
30     protocol    = "-1"
31     cidr_blocks = ["0.0.0.0/0"]
32   }
33 }

lab01 > terraform > modules > security_group > main.tf
11 resource "aws_security_group" "public_ec2" {
12   tags = {
13     Name = "${var.name_prefix}-public-ec2-sg"
14   }
15 }
16
17 # --- Private EC2 Security Group ---
18 resource "aws_security_group" "private_ec2" {
19   name        = "${var.name_prefix}-private-ec2-sg"
20   description = "Allow SSH from Public EC2 SG only"
21   vpc_id      = var.vpc_id
22
23   # SSH only from Public EC2 Security Group
24   ingress {
25     description = "SSH from Public EC2 SG"
26     from_port   = 22
27     to_port     = 22
28     protocol    = "tcp"
29     security_groups = [aws_security_group.public_ec2.id]
30   }
31
32   # Allow all outbound (needed for NAT Gateway → Internet)
33   egress {
34     description = "Allow all outbound"
35     from_port   = 0
36     to_port     = 0
37     protocol    = "-1"
38     cidr_blocks = ["0.0.0.0/0"]
39   }
40
41   tags = {
42     Name = "${var.name_prefix}-private-ec2-sg"
43   }
44 }
```

Phân tích kỹ thuật: Security Group (SG) hoạt động ở tầng giao diện mạng (ENI) của từng máy ảo. Module này được thiết kế dựa trên nguyên tắc "Đặc quyền tối thiểu" (Least Privilege) để tối ưu hóa an ninh:

- **Public SG:** Ở chiều Inbound, hệ thống chặn mọi truy cập công khai và chỉ mở đúng Port 22 (giao thức SSH) cho duy nhất IP tĩnh của người quản trị (thông qua biến `var.my_ip`). Chiều Outbound mở hoàn toàn để máy chủ có thể phản hồi request.
- **Private SG (Tham chiếu chéo):** Không sử dụng địa chỉ IP làm source đầu vào. Thay vào đó, Inbound rule được thiết lập để nhận kết nối từ một ID Security Group khác (`security_groups = [aws_security_group.public_ec2.id]`). Cơ chế tham chiếu chéo này trên AWS đảm bảo rằng chỉ có các máy ảo đang mang trên mình bộ giáp "Public SG"

mới có quyền kết nối vào máy ảo mang "Private SG". Đây là cốt lõi để xây dựng mô hình trạm trung chuyển Bastion Host an toàn nhất.

1.3.7. Module EC2

Khởi tạo Virtual Private Cloud với dải mạng 10.0.0.0/16. Các tham số `enable_dns_support` và `enable_dns_hostnames` được bật để các EC2 instances tự động được gán DNS names, thuận tiện cho việc định danh trong mạng nội bộ.

🔗 Mã nguồn main.tf:

```
lab01 > terraform > modules > ec2 > main.tf
1 | # --- Public EC2 Instance ---
2 | resource "aws_instance" "public" {
3 |     ami                = var.ami_id
4 |     instance_type      = var.instance_type
5 |     subnet_id          = var.public_subnet_id
6 |     vpc_security_group_ids = [var.public_sg_id]
7 |     key_name           = var.key_name
8 |     associate_public_ip_address = true
9 |
10 |     tags = {
11 |         Name = "${var.name_prefix}-public-ec2"
12 |         Role = "Public"
13 |     }
14 | }
15 |
16 | # --- Private EC2 Instance ---
17 | resource "aws_instance" "private" {
18 |     ami                = var.ami_id
19 |     instance_type      = var.instance_type
20 |     subnet_id          = var.private_subnet_id
21 |     vpc_security_group_ids = [var.private_sg_id]
22 |     key_name           = var.key_name
23 |     associate_public_ip_address = false
24 |
25 |     tags = {
26 |         Name = "${var.name_prefix}-private-ec2"
27 |         Role = "Private"
28 |     }
29 | }
```

Phân tích kỹ thuật: Module EC2 đảm nhiệm việc khởi tạo các máy chủ điện toán dựa trên nền tảng mạng và bảo mật đã thiết lập ở các module trước. Hai máy chủ được thiết kế với vai trò và mức độ tiếp xúc Internet hoàn toàn khác biệt:

- Public EC2 (Đóng vai trò Web Server / Bastion Host):

- Được định tuyến vào Public Subnet (`var.public_subnet_id`) và gắn lớp giáp bảo vệ là Public Security Group.
- Điểm mấu chốt là cờ `associate_public_ip_address = true` buộc hệ thống AWS phải cấp phát một địa chỉ IP công cộng. Nhờ đó, người quản trị có thể trực tiếp dùng giao thức SSH từ máy local để điều khiển instance này.

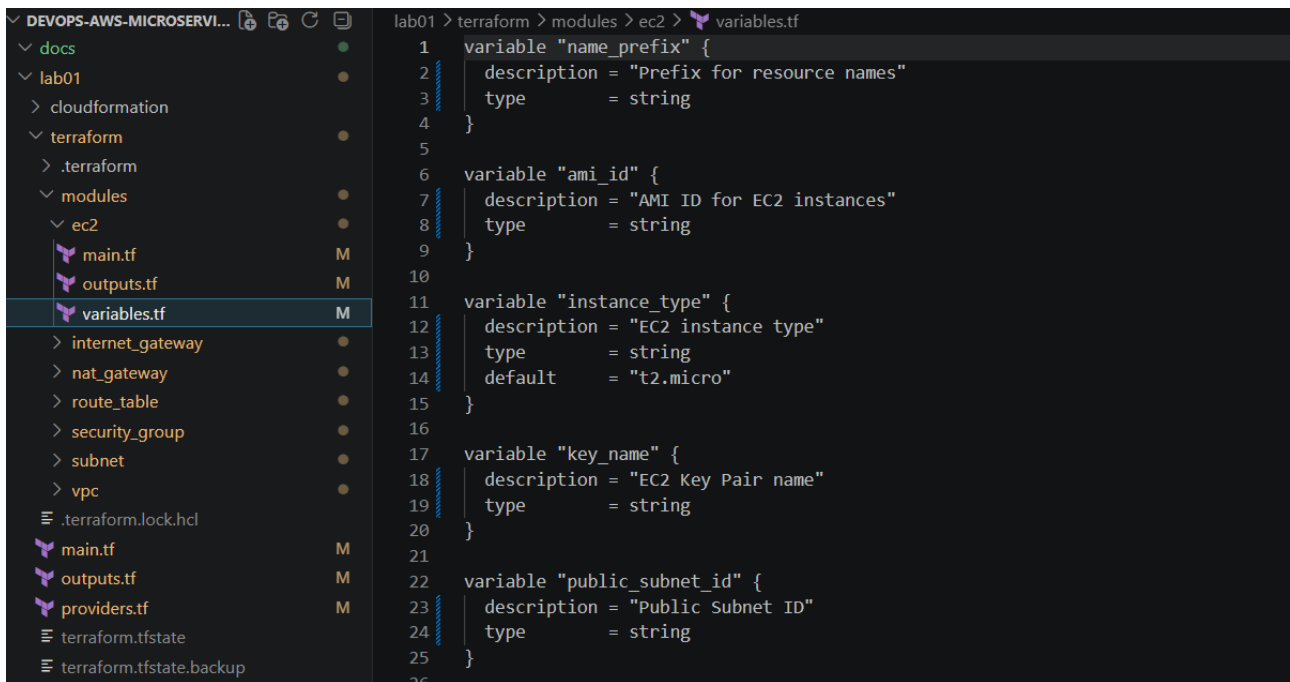
- Private EC2 (Đóng vai trò Backend / Database):

- Được đặt sâu bên trong vùng mạng nội bộ Private Subnet (`var.private_subnet_id`).

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

- Thuộc tính **associate_public_ip_address = false** được thiết lập nhằm chặn hoàn toàn việc gán Public IP. Máy chủ này "tàng hình" trước Internet; nó chỉ có thể được truy cập thông qua Private IP nội bộ (10.0.2.71) từ máy Public EC2 ở trên. Đây là cách triển khai chuẩn theo mô hình phòng thủ chiều sâu (Defense in Depth).
- **Tính tái sử dụng cao:** Các thông số lỗi như hệ điều hành (ami), cấu hình phần cứng (instance_type), và khóa bảo mật (key_name) đều được truyền dưới dạng biến (var). Cách code này giúp tránh việc "hard-code", cho phép module dễ dàng tái sử dụng nếu dự án cần triển khai thêm các cấu hình máy chủ lớn hơn mà không cần viết lại mã nguồn.
⇔ Để hiện thực hóa tính linh hoạt này, toàn bộ các tham số đầu vào được định nghĩa độc lập trong file **variables.tf** của module.

🔗 Mã nguồn variables.tf:



```
1 variable "name_prefix" {
2   | description = "Prefix for resource names"
3   | type       = string
4 }
5
6 variable "ami_id" {
7   | description = "AMI ID for EC2 instances"
8   | type       = string
9 }
10
11 variable "instance_type" {
12   | description = "EC2 instance type"
13   | type       = string
14   | default    = "t2.micro"
15 }
16
17 variable "key_name" {
18   | description = "EC2 Key Pair name"
19   | type       = string
20 }
21
22 variable "public_subnet_id" {
23   | description = "Public Subnet ID"
24   | type       = string
25 }
26
```

1.4.Quy trình triển khai

Quá trình triển khai hạ tầng được thực hiện tự động thông qua các câu lệnh cốt lõi của Terraform.

- Bước 1: Khởi tạo Workspace

```
terraform init
```

Lệnh này được sử dụng để tải các plugin cần thiết (như AWS Provider) và thiết lập môi trường làm việc ban đầu cho Terraform.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

```
PS D:\StudywithAlex\UIT\HK6\NT548\LAB\devops-aws-microservices\lab01\terraform> terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

- ➔ Terraform đã tự động nhận diện cấu hình, tải về thư viện hashicorp/aws phiên bản v5.100.0 và khởi tạo thành công file khóa terraform.lock.hcl.

- Bước 2: Định dạng và kiểm tra mã nguồn

terraform fmt & terraform validate

Trước khi triển khai, mã nguồn cần được chuẩn hóa format và kiểm tra tính hợp lệ của cú pháp.

```
● PS D:\StudywithAlex\UIT\HK6\NT548\LAB\devops-aws-microservices\lab01\terraform> terraform fmt
terraform.tfvars
● PS D:\StudywithAlex\UIT\HK6\NT548\LAB\devops-aws-microservices\lab01\terraform> terraform validate
Success! The configuration is valid.
```

- ➔ Lệnh fmt đã phát hiện và tự động căn chỉnh lại file terraform.tfvars.
- ➔ Lệnh validate xác nhận toàn bộ các liên kết giữa các module và biến đầu vào không có lỗi logic, trả về thông báo Success!.

- Bước 3: Lập kế hoạch triển khai

terraform plan

Lệnh này giúp người quản trị "xem trước" những tài nguyên nào sẽ được tạo, sửa, hoặc xóa trên AWS để tránh rủi ro.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

Plan: 15 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ internet_gateway_id = (known after apply)
+ nat_gateway_id      = (known after apply)
+ private_ec2_private_ip = (known after apply)
+ private_subnet_id   = (known after apply)
+ public_ec2_public_ip = (known after apply)
+ public_subnet_id     = (known after apply)
+ ssh_to_private_ec2   = (known after apply)
+ ssh_to_public_ec2    = (known after apply)
+ vpc_id               = (known after apply)
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

- ➔ Terraform tính toán và vạch ra lộ trình (execution plan) gồm việc khởi tạo mới hoàn toàn 15 tài nguyên (instances, security groups, gateways,...), không có tài nguyên nào bị xóa hay thay đổi.

- Bước 4: Thực thi cấp phát tài nguyên

terraform apply

Sau khi xác nhận plan hợp lý, lệnh apply được gọi để tự động gọi AWS API và xây dựng hạ tầng.

```
PS D:\StudywithAlex\UIT\HK6\NT548\LAB\devops-aws-microservices\lab01\terraform> terraform apply -auto-approve
data.aws_ami.amazon_linux_2[0]: Reading...
module.vpc.aws_vpc.main: Refreshing state... [id=vpc-01ea198cb9a83707e]
data.aws_ami.amazon_linux_2[0]: Read complete after 1s [id=ami-0e8bf7e1d1f339c74]
module.internet_gateway.aws_internet_gateway.main: Refreshing state... [id=igw-008d03ee3f7508577]
module.security_group.aws_security_group.public_ec2: Refreshing state... [id=sg-0c26cf1cfafb3efcc]
module.subnet.aws_subnet.private: Refreshing state... [id=subnet-08a00044de5292751]
module.security_group.aws_default_security_group.default: Refreshing state... [id=sg-0cfff0ac16dca8800]
module.subnet.aws_subnet.public: Refreshing state... [id=subnet-0c68cf1186ab19f6d]
module.route_table.aws_route_table.public: Refreshing state... [id=rtb-05e72e38a1f91b739]
module.nat_gateway.aws_eip.nat: Refreshing state... [id=eipalloc-0e52588c618f742e4]
module.security_group.aws_security_group.private_ec2: Refreshing state... [id=sg-090052b79c329aa06]
module.route_table.aws_route_table_association.public: Refreshing state... [id=rtbassoc-09e949ab4ac73e1ff]
module.nat_gateway.aws_nat_gateway.main: Refreshing state... [id=nat-09f97ea904c129b47]
module.route_table.aws_route_table.private: Refreshing state... [id=rtb-0c1c0e1101f94fe7d]
module.route_table.aws_route_table_association.private: Refreshing state... [id=rtbassoc-065426d5d7301cfdcf]
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```
internet_gateway_id = "igw-008d03ee3f7508577"
nat_gateway_id      = "nat-09f97ea904c129b47"
private_ec2_private_ip = "10.0.2.71"
private_subnet_id   = "subnet-08a00044de5292751"
public_ec2_public_ip = "13.250.125.98"
public_subnet_id     = "subnet-0c68cf1186ab19f6d"
ssh_to_private_ec2   = "ssh -i ~/.ssh/nt548-key.pem ec2-user@10.0.2.71"
ssh_to_public_ec2    = "ssh -i ~/.ssh/nt548-key.pem ec2-user@13.250.125.98"
vpc_id               = "vpc-01ea198cb9a83707e"
PS D:\StudywithAlex\UIT\HK6\NT548\LAB\devops-aws-microservices\lab01\terraform>
```

➔ Quá trình khởi tạo thành công (Apply complete!). Một số tài nguyên được tạo song song, trong khi các tài nguyên như nat_gateway phải chờ đợi hoàn tất

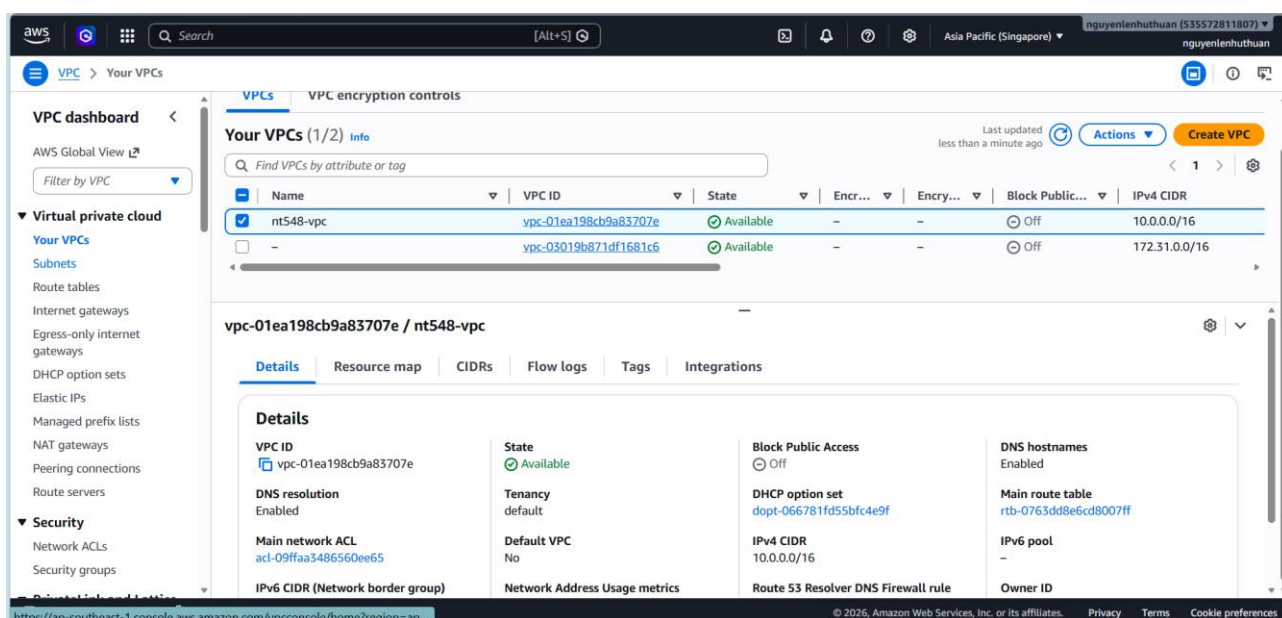
➔ Bảng **Outputs** trả về các thông số sinh lời quan trọng để quản trị:

- vpc_id = vpc-01ea198cb9a83707e
- Máy ảo Public được cấp IP: 13.250.125.98
- Máy ảo Private được cấp IP nội bộ: 10.0.2.71

1.5. Xác minh kết quả trên giao diện AWS (AWS Console)

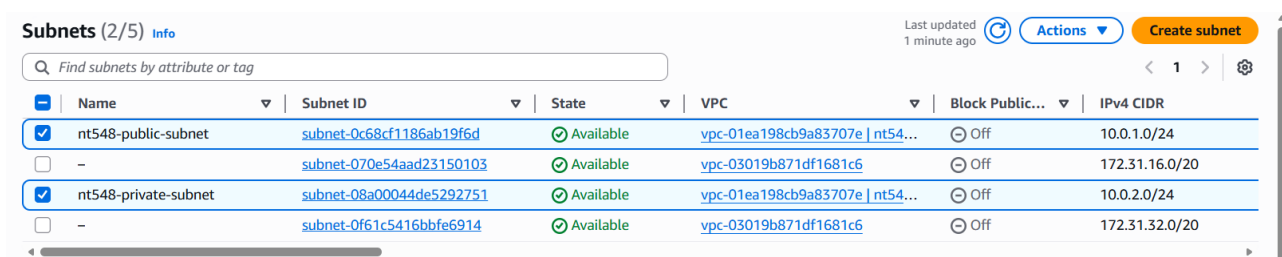
Sau khi quá trình tự động hóa hoàn tất, tiến hành kiểm tra chéo (cross-check) trên AWS Console để xác nhận hạ tầng thực tế đã khớp với mã nguồn.

- **Kiểm tra VPC:** Tại dịch vụ VPC, tra cứu vpc-01ea198cb9a83707e đã tạo.



➔ **Phân tích:** Mạng ảo đã được cấp phát đúng với dải CIDR 10.0.0.0/16 khai báo trong biến đầu vào.

- **Kiểm tra phân mảnh mạng (Subnets):** Tại mục Subnets, hiển thị danh sách các mạng con thuộc VPC trên.



Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

Subnets (1/5) Info

Last updated 3 minutes ago

Actions

Create subnet

Find subnets by attribute or tag

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☒ nt548-public-subnet	subnet-0c68cf1186ab19f6d	Available	vpc-01ea198cb9a83707e nt54...	Off	10.0.1.0/24
☐ -	subnet-070e54aad23150103	Available	vpc-03019b871df1681c6	Off	172.31.16.0/20
☐ nt548-private-subnet	subnet-08a00044de5292751	Available	vpc-01ea198cb9a83707e nt54...	Off	10.0.2.0/24
☐ -	subnet-0f61c5416bbfe6914	Available	vpc-03019b871df1681c6	Off	172.31.32.0/20

subnet-0c68cf1186ab19f6d / nt548-public-subnet

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Details

Subnet ID

subnet-0c68cf1186ab19f6d

Subnet ARN

arn:aws:ec2:ap-southeast-1:535572811807:subnet/subnet-0c68cf1186ab19f6d

State

Available

Block Public Access

Off

IPv4 CIDR

10.0.1.0/24

Availability Zone

ap-se1-az2 (ap-southeast-1a)

IPv6 CIDR

-

IPv6 CIDR association ID

-

Network ACL

acl-085f7189558a655

Available IPv4 addresses

249

VPC

vpc-01ea198cb9a83707e | nt548-vpc

Route table

rtb-05e72e38a1f91b739 | nt548-public-rt

Network border group

ap-southeast-1

Auto-assign public IPv4 address

-

➔ **Phân tích:** Hai Subnet đã được cắt thành công. Public Subnet mang dải 10.0.1.0/24 và Private Subnet mang dải 10.0.2.0/24, cả hai đều nằm chung trong cùng một Availability Zone (ap-southeast-1a).

- Kiểm tra Gateway (Internet Gateway & NAT Gateway)

aws

Search

[Alt+S]

Asia Pacific (Singapore)

nguyenienhuthuan (535572811807)

nguyenienhuthuan

VPC > Internet gateways

VPC dashboard <

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

Internet gateways (1/2) Info

Find internet gateways by attribute or tag

Actions

Create internet gateway

Name	Internet gateway ID	State	VPC ID	Owner
☒ nt548-igw	igw-008d03ee3f7508577	Attached	vpc-01ea198cb9a83707e nt548-vpc	535572811807
☐ -	igw-0edc31c9a2d7ea2fb	Attached	vpc-03019b871df1681c6	535572811807

igw-008d03ee3f7508577 / nt548-igw

Tags (5)

Search tags

Key	Value
Name	nt548-igw
Environment	dev
Owner	nt548-group-03
ManagedBy	Terraform
Project	NT548-Lab01

Manage tags

➔ **Internet Gateway** (igw-008d03ee3f7508577) đã ở trạng thái Attached vào VPC nt548.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

NAT gateways (1/1) Info

Find NAT gateways by attribute or tag

Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route table ID
nt548-nat-gw	nat-09f97ea904c129b47	Public	Available	-	Zonal	-

nat-09f97ea904c129b47 / nt548-nat-gw

Details

NAT gateway ID

nat-09f97ea904c129b47

NAT gateway ARN

arn:aws:ec2:ap-southeast-1:535572811807:natgateway/nat-09f97ea904c129b47

VPC

vpc-01ea198cb9a83707e / nt548-vpc

Connectivity type

Public

Primary public IPv4 address

47.128.203.117

Subnet

subnet-0c68cf1186ab19f6d / nt548-public-subnet

State

Available

Primary private IPv4 address

10.0.1.84

Created

Monday, June 1, 2026 at 15:06:36 GMT+7

State message

-

Primary network interface ID

eni-0c51db44c00b9f646

Deleted

-

➔ NAT Gateway (nat-09f97ea904c129b47) hiển thị trạng thái Available và đã được AWS gán cứng một Public IPv4 address tĩnh.

- Kiểm tra định tuyến (Route Tables) Tại mục Route Tables, kiểm tra tab Routes của từng bảng.

Route tables (1/4) Info

Last updated 6 minutes ago

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
-	rtb-0763dd8e6cd8007ff	-	-	Yes	vpc-01ea198cb9a83707e nt548-vpc
nt548-public-rt	rtb-05e72e38a1f91b739	subnet-0c68cf1186ab19f6d	-	No	vpc-01ea198cb9a83707e nt548-vpc
nt548-private-rt	rtb-0c1c0e1101f94fe7d	subnet-08a00044de5292...	-	No	vpc-01ea198cb9a83707e nt548-vpc

rtb-05e72e38a1f91b739 / nt548-public-rt

Details

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-008d03ee3f7508577	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

nt548-private-rt

rtb-0c1c0e1101f94fe7d / nt548-private-rt

Details

Routes

Subnet associations

Edge associations

Route propagation

Tags

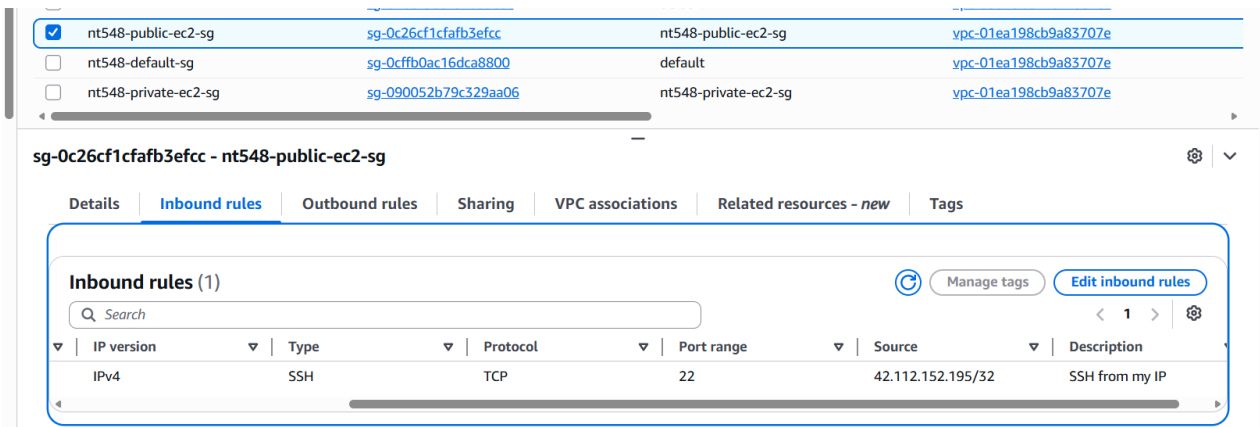
Routes (2)

Filter routes

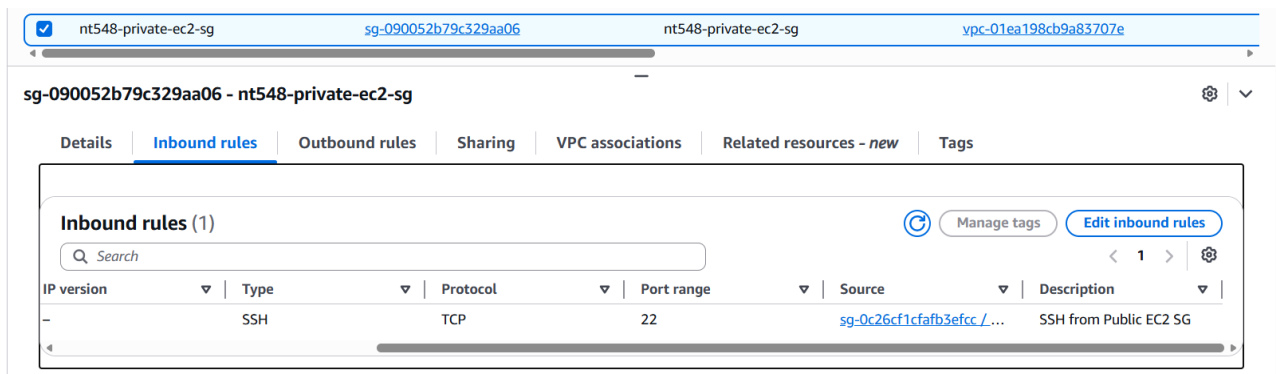
Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-09f97ea904c129b47	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

➔ Các quy tắc đã được gán chính xác: Public Route Table đẩy luồng dữ liệu 0.0.0.0/0 qua IGW, còn Private Route Table đẩy gói tin qua NAT.

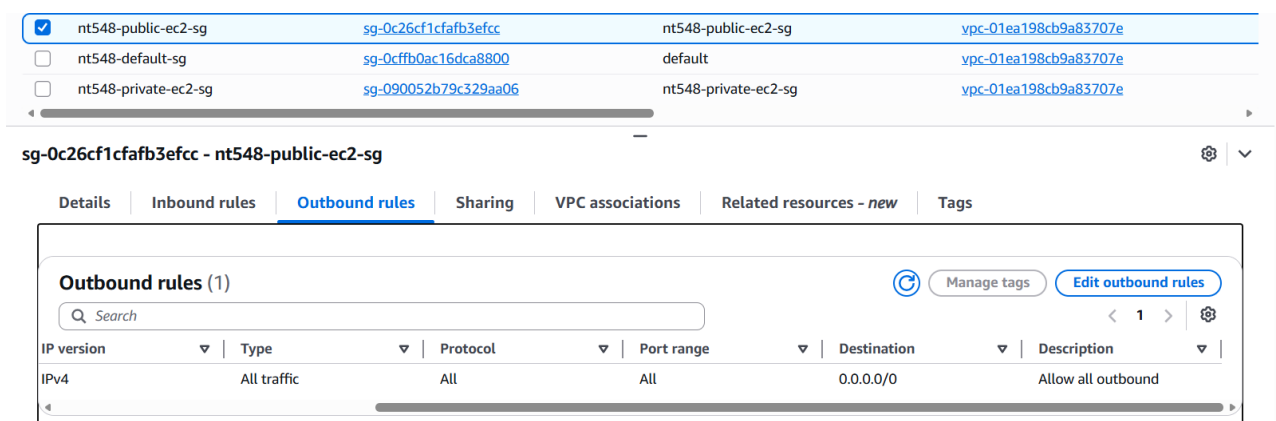
- **Kiểm tra cấu hình tường lửa (Security Groups):** Tại dịch vụ VPC, truy cập vào mục **Security Groups**. Tiến hành kiểm tra các quy tắc Inbound của 2 nhóm tường lửa chính đã được cấu hình.



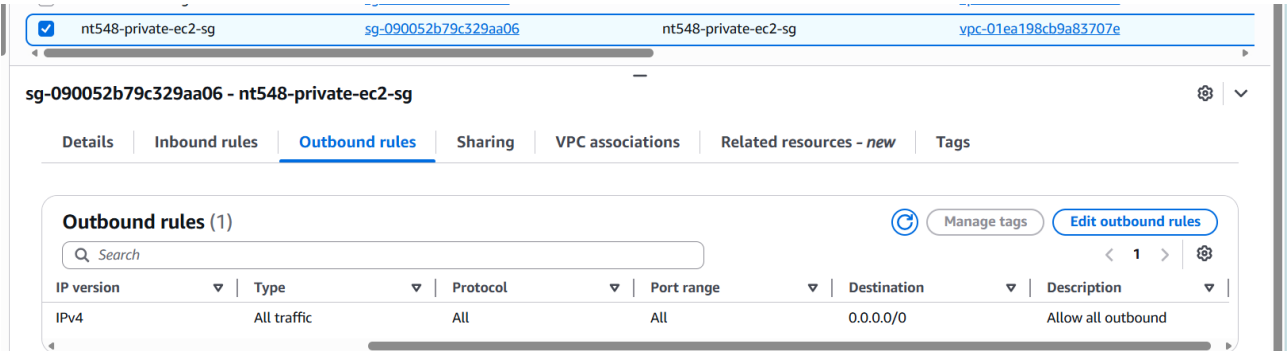
- ➔ Đối với **Public SG**: Hệ thống chỉ mở duy nhất port 22 (SSH) và giới hạn cứng dải IP được phép truy cập (42.112.152.195/32). Mọi IP khác trên Internet cố gắng kết nối đều sẽ bị rớt gói tin (drop).



- ➔ Đối với **Private SG**: Port 22 được mở, nhưng Source không phải là một địa chỉ IP mà là ID của nt548-public-ec2-sg (sg-0c26cf1cfa3b3efcc). Điều này minh chứng thành công cơ chế tham chiếu chéo (Cross-referencing) của AWS: chỉ những máy ảo nào mang bộ giáp Public SG mới có quyền "gõ cửa" máy ảo Private.

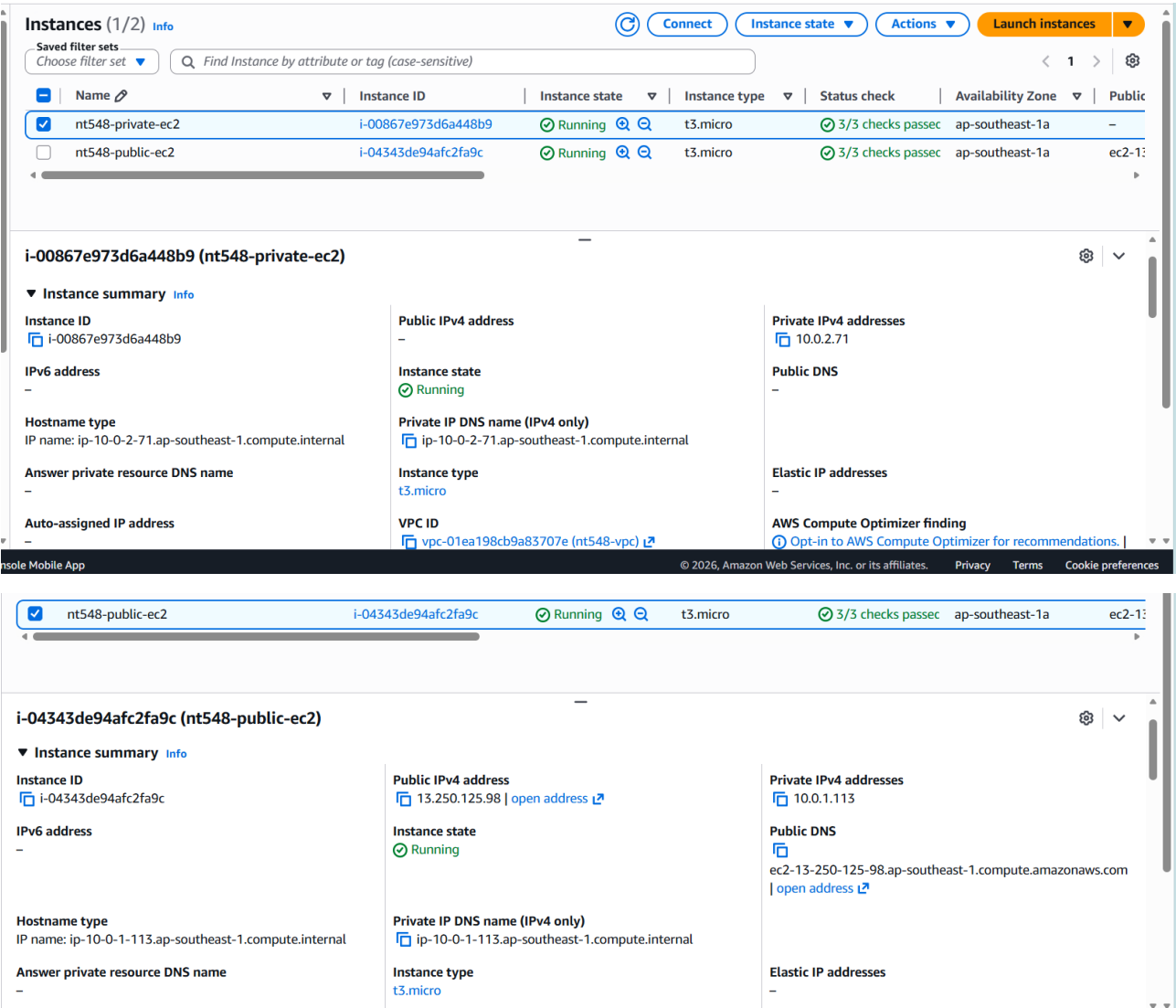


Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS



➔ Chiều Outbound của cả hai SG đều được mở 0.0.0.0/0 để đảm bảo máy chủ có thể trả lời các request.

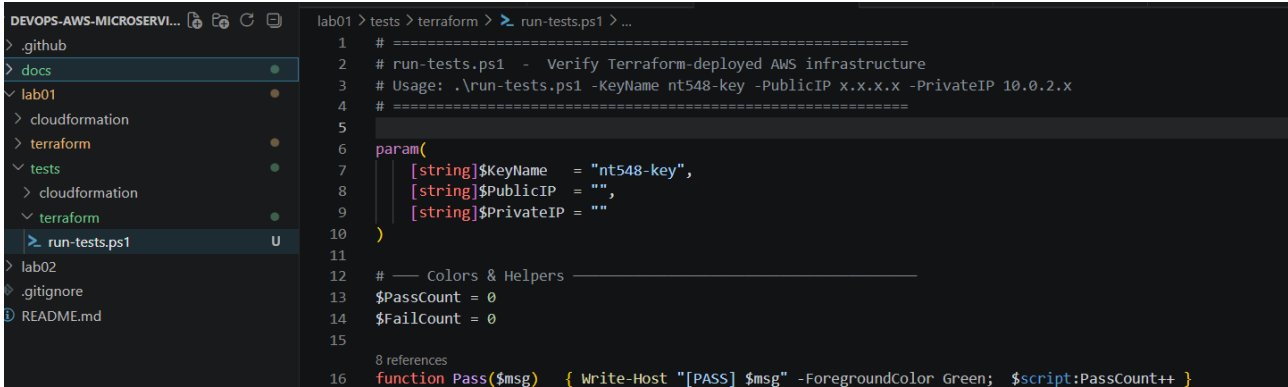
- Kiểm tra máy chủ ảo điện toán (EC2 Instances) Tại dịch vụ EC2, kiểm tra danh sách Instances đang chạy.



➔ **Phân tích:** Trạng thái hệ thống hoàn toàn khớp với thiết kế. Máy ảo Public được AWS gán IP công cộng (13.250.125.98), trong khi máy Private bị chặn gán IP, đảm bảo sự cô lập an toàn trên môi trường Cloud.

1.6. Test cases và xác minh

Để tự động hóa quá trình xác minh (Validation) toàn bộ hạ tầng vừa triển khai, em xây dựng script **run-tests.ps1** thực thi 7 test cases độc lập, tự động đọc IP từ Terraform output.

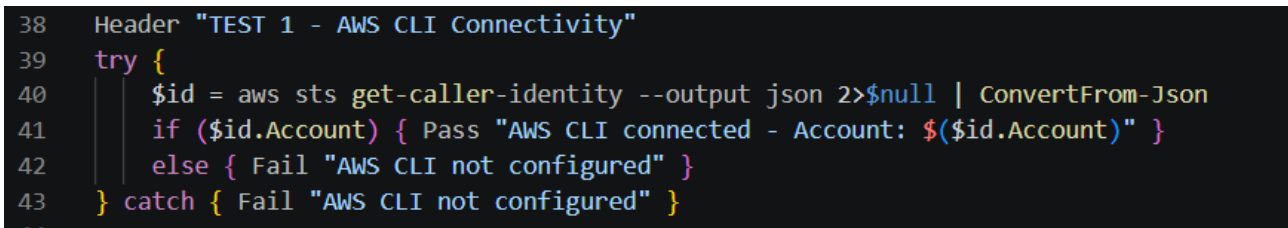


```
lab01 > tests > terraform > run-tests.ps1 > ...
1 # =====
2 # run-tests.ps1 - Verify Terraform-deployed AWS infrastructure
3 # Usage: .\run-tests.ps1 -KeyName nt548-key -PublicIP x.x.x.x -PrivateIP 10.0.2.x
4 # =====
5
6 param(
7     [string]$KeyName = "nt548-key",
8     [string]$PublicIP = "",
9     [string]$PrivateIP = ""
10 )
11
12 # --- Colors & Helpers ---
13 $PassCount = 0
14 $FailCount = 0
15
16 8 references
17 function Pass($msg) { Write-Host "[PASS] $msg" -ForegroundColor Green; $script:PassCount++ }
```

Dưới đây là phân tích chi tiết mục đích và cơ chế của từng test case:

- Test 1 – AWS CLI Connectivity

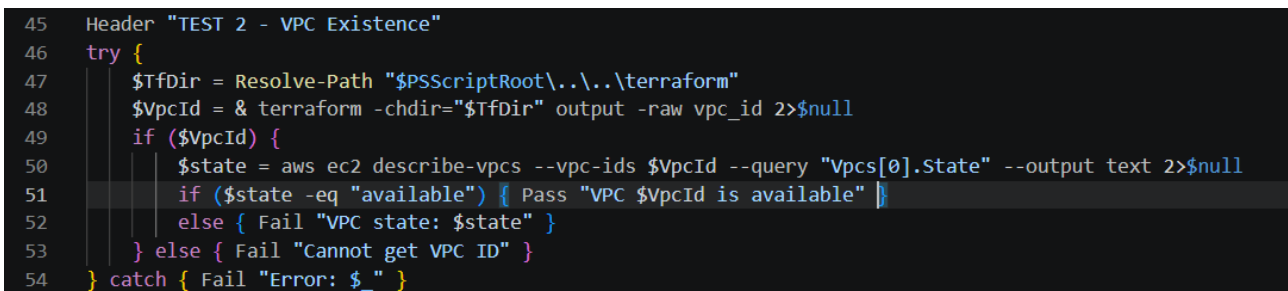
- **Mục đích:** Xác minh credentials AWS đã được cấu hình đúng.
- **Cơ chế:** Gọi aws sts get-caller-identity, nếu trả về Account ID → PASS.



```
38 Header "TEST 1 - AWS CLI Connectivity"
39 try {
40     $id = aws sts get-caller-identity --output json 2>$null | ConvertFrom-Json
41     if ($id.Account) { Pass "AWS CLI connected - Account: $($id.Account)" }
42     else { Fail "AWS CLI not configured" }
43 } catch { Fail "AWS CLI not configured" }
```

- Test 2 – VPC Existence

- **Mục đích:** Xác nhận VPC tồn tại và ở trạng thái available.
- **Cơ chế:** Lấy vpc_id từ Terraform output, query AWS CLI kiểm tra state.



```
45 Header "TEST 2 - VPC Existence"
46 try {
47     $TfDir = Resolve-Path "$PSScriptRoot\..\..\terraform"
48     $VpcId = & terraform -chdir="$TfDir" output -raw vpc_id 2>$null
49     if ($VpcId) {
50         $state = aws ec2 describe-vpcs --vpc-ids $VpcId --query "Vpcs[0].State" --output text 2>$null
51         if ($state -eq "available") { Pass "VPC $VpcId is available" }
52         else { Fail "VPC state: $state" }
53     } else { Fail "Cannot get VPC ID" }
54 } catch { Fail "Error: $_" }
```

- Test 3 – Public EC2 Reachability

- **Mục đích:** Xác nhận IGW và Public Security Group (port 22 mở cho IP máy local) hoạt động.
- **Cơ chế:** SSH trực tiếp từ máy local vào Public IP. Kết nối thành công = luồng Internet → IGW → Public Subnet → EC2 thông suốt.

```
56 Header "TEST 3 - SSH to Public EC2"
57 try {
58     $r = & ssh @SO "ec2-user@$PublicIP" "echo ok" 2>$null
59     if ($r -match "ok") { Pass "SSH to Public EC2 ($PublicIP) succeeded" }
60     else { Fail "SSH to Public EC2 failed - check key and Security Group" }
61 } catch { Fail "SSH failed: $_" }
```

- Test 4 – Public EC2 Internet Access

- **Mục đích:** Xác nhận outbound traffic từ Public Subnet ra Internet.
- **Cơ chế:** Từ bên trong Public EC2, chạy curl <https://checkip.amazonaws.com>, kết quả trả về đúng Public IP của EC2.

```
63 Header "TEST 4 - Public EC2 Internet Access"
64 try {
65     $ip = & ssh @SO "ec2-user@$PublicIP" "curl -s --max-time 5 https://checkip.amazonaws.com" 2>$null
66     if ($ip) { Pass "Public EC2 Internet OK - external IP: $($ip.Trim())" }
67     else { Fail "Public EC2 cannot reach Internet" }
68 } catch { Fail "Test failed: $_" }
```

- Test 5 – Private EC2 Reachability (Bastion Host)

- **Mục đích:** Kiểm tra kết nối nội bộ Public → Private Subnet và Private Security Group hoạt động đúng.
- **Cơ chế:** Copy file .pem lên Public EC2, sau đó thực hiện SSH jump (lồng nhau) từ Public EC2 vào Private EC2 qua private IP.

```
70 Header "TEST 5 - SSH Jump to Private EC2"
71 try {
72     & scp @SO $KeyPath "ec2-user@$PublicIP:~/.ssh/id_rsa" 2>$null
73     & ssh @SO "ec2-user@$PublicIP" "chmod 400 ~/.ssh/id_rsa" 2>$null
74     $r = & ssh @SO "ec2-user@$PublicIP" "ssh -i ~/.ssh/id_rsa -o StrictHostKeyChecking=no -o ConnectTimeout=10 ec2-user@$PrivateIP 'echo ok'" 2>$null
75     if ($r -match "ok") { Pass "SSH Public -> Private EC2 ($PrivateIP) succeeded" }
76     else { Fail "SSH jump to Private EC2 failed" }
77 } catch { Fail "Test failed: $_" }
```

- Test 6 – Private EC2 Internet via NAT Gateway

- **Mục đích:** Xác minh NAT Gateway và Private Route Table hoạt động — đây là test quan trọng nhất.
- **Cơ chế:** Điều khiển Private EC2 (qua SSH jump) chạy curl ra Internet. NAT Gateway dịch IP nội bộ 10.0.2.x thành Elastic IP, curl trả về EIP → PASS.

```
79 Header "TEST 6 - Private EC2 Internet via NAT"
80 try {
81     $r = & ssh @SO "ec2-user@$PublicIP" "ssh -i ~/.ssh/id_rsa -o StrictHostKeyChecking=no ec2-user@$PrivateIP 'curl -s --max-time 10 https://checkip.amazonaws.com'" 2>$null
82     if ($r) { Pass "Private EC2 NAT OK - external IP: $($r.Trim())" }
83     else { Fail "Private EC2 cannot reach Internet via NAT" }
84 } catch { Fail "Test failed: $_" }
```

- Test 7 – Private EC2 Isolation (Security Verification)

- **Mục đích:** Khẳng định Private EC2 hoàn toàn cách ly khỏi Internet trực tiếp.
- **Cơ chế:** Cố tình SSH thẳng từ máy local vào Private IP — kết nối **bắt buộc phải timeout/thất bại** thì mới PASS, chứng minh không có lỗ hổng bảo mật.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

```
86 Header "TEST 7 - Private EC2 NOT reachable from Internet"
87 try {
88     $r = & ssh -i $KeyPath -o StrictHostKeyChecking=no -o ConnectTimeout=5 -o BatchMode=yes "ec2-user@$PrivateIP" "echo ok" 2>$null
89     if ($r -match "ok") { Fail "Private EC2 is directly accessible - security risk!" }
90     else { Pass "Private EC2 is NOT directly accessible from Internet (correct)" }
91 } catch { Pass "Private EC2 is NOT directly accessible from Internet (correct)" }
```

Lệnh thực thi kiểm thử

- Để khởi chạy quy trình kiểm tra trên môi trường Windows, mở terminal tại thư mục chứa file test và thực thi lệnh sau (sử dụng cờ -ExecutionPolicy Bypass để cấp quyền chạy script cho PowerShell):

```
powershell -ExecutionPolicy Bypass -File ".\run-tests.ps1" -KeyName nt548-key -PublicIP 18.143.179.23 -PrivateIP 10.0.2.116
```

```
PS D:\StudywithAlex\UIT\HK6\NT548\LAB\devops-aws-microservices\lab01\tests\terraform> powershell -ExecutionPolicy Bypass -File ".\run-tests.ps1" -KeyName nt548-key -PublicIP 18.143.179.23 -PrivateIP 10.0.2.116
[INFO] Public EC2 IP : 18.143.179.23
[INFO] Private EC2 IP : 10.0.2.116
[INFO] Key Path      : C:\Users\Admin\.ssh\nt548-key.pem

=====
TEST 1 - AWS CLI Connectivity
=====
[PASS] AWS CLI connected - Account: 535572811807

=====
TEST 2 - VPC Existence
=====
[PASS] VPC vpc-09f8b8c8d294b2b09 is available

=====
TEST 3 - SSH to Public EC2
=====
[PASS] SSH to Public EC2 (18.143.179.23) succeeded

=====
TEST 4 - Public EC2 Internet Access
=====
[PASS] Public EC2 Internet OK - external IP: 18.143.179.23

=====
TEST 5 - SSH Jump to Private EC2
=====
nt548-key.pem 100% 1678 39.0KB/s 00:00
[PASS] SSH Public -> Private EC2 (10.0.2.116) succeeded

=====
TEST 6 - Private EC2 Internet via NAT
=====
[PASS] Private EC2 NAT OK - external IP: 13.228.112.125

=====
TEST 7 - Private EC2 NOT reachable from Internet
=====
[PASS] Private EC2 is NOT directly accessible from Internet (correct)

=====
SUMMARY
=====
Passed: 7 | Failed: 0
All tests passed! Infrastructure is healthy.
PS D:\StudywithAlex\UIT\HK6\NT548\LAB\devops-aws-microservices\lab01\tests\terraform>
```

➔ **Đánh giá kết quả Test:** Tổng cộng 7/7 bài kiểm tra đều vượt qua thành công (Passed: 7). Đặc biệt khi đối chiếu giữa **Test 4** và **Test 6**, ta thấy rõ sự khác biệt trong định tuyến:

- Ở Test 4, máy ảo Public trực tiếp đi ra Internet nên địa chỉ trả về chính là Public IP của nó (18.143.179.23).
- Ở Test 6, máy ảo Private hoàn toàn không có Public IP, gói tin của nó bắt buộc phải chạy qua NAT Gateway. Do đó, địa chỉ IP giao tiếp với Internet hiển thị là 13.228.112.125 (chính là Elastic IP đã cấp phát cho NAT).

➔ Kết quả này khẳng định tuyệt đối hạ tầng AWS đã được cấp phát tự động không chỉ đúng số lượng tài nguyên mà còn được liên kết mạng (Network Routing) và bảo mật (Security Groups) chính xác theo thiết kế kiến trúc ban đầu.

Câu 2 — Triển khai hạ tầng AWS bằng CloudFormation

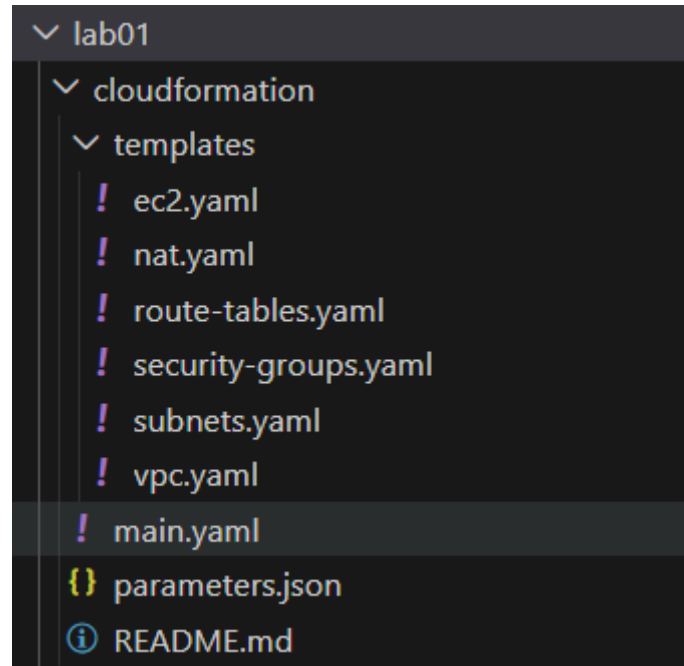
2.1. Mục tiêu và kiến trúc

Mục tiêu: Kiến trúc CloudFormation được chia nhỏ thành các Nested Stacks. Mỗi file YAML đóng vai trò như một module độc lập, nhận biến qua khối Parameters và xuất kết quả qua khối Outputs. Dưới đây là phân tích chi tiết logic cốt lõi của từng template.

Bảng resources được tạo:

Resource	Số lượng	CIDR / Thông tin chi tiết	Chức năng chính
VPC	1	10.0.0.0/16	VPC chứa toàn bộ hạ tầng của bài lab.
Public Subnet	1	10.0.1.0/24	Phân mạng chứa các tài nguyên có thể tiếp cận từ Internet như Public EC2, NAT Gateway, ...
Private Subnet	1	10.0.2.0/24	Phân mạng chứa tài nguyên nội bộ, hoàn toàn không có Public IP - Private EC2.
Internet Gateway	1	-	Cổng giao tiếp mạng hai chiều giữa VPC và môi trường Internet bên ngoài.
NAT Gateway	1	Kèm 1 Elastic IP	Nằm ở vùng Public, cấp quyền cho Private EC2 đi ra Internet.
Route Tables	2	Public RT, Private RT	Bảng định tuyến, đóng vai trò điều hướng traffic cho các Subnet tương ứng.
Security Groups	2	Public SG, Private SG	Tường lửa ảo kiểm soát luồng Inbound/Outbound.
EC2 Instances	2	t3.micro (Free Tier)	Máy chủ ảo tính toán, bao gồm 1 Public EC2 và 1 Private EC2.

2.2. Cấu trúc thư mục CloudFormation



2.3. Triển khai các template

2.3.1. Template VPC (vpc.yaml)

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'VPC + Internet Gateway'

Parameters:
  NamePrefix:
    Type: String
  VpcCidr:
    Type: String

Resources:
  Vpc:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: !Ref VpcCidr
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-vpc'

  InternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties:
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-igw'

  IgwAttachment:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties:
      VpcId: !Ref Vpc
      InternetGatewayId: !Ref InternetGateway

Outputs:
  VpcId:
    Value: !Ref Vpc
  InternetGatewayId:
    Value: !Ref InternetGateway
```

Phân tích kỹ thuật: Sử dụng tài nguyên vật lý `AWS::EC2::VPCGatewayAttachment` để liên kết chặt chẽ Internet Gateway vào VPC. Hàm `!Sub` được áp dụng để nối chuỗi linh hoạt cho các thẻ Tags, giúp chuẩn hóa quy tắc đặt tên tài nguyên trên toàn hệ thống.

2.3.2. Template Subnets (subnets.yaml)

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Public + Private Subnets'

Parameters:
  NamePrefix:
    Type: String
  VpcId:
    Type: AWS::EC2::VPC::Id
  PublicSubnetCidr:
    Type: String
  PrivateSubnetCidr:
    Type: String
  AvailabilityZone:
    Type: AWS::EC2::AvailabilityZone::Name

Resources:
  PublicSubnet:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PublicSubnetCidr
      AvailabilityZone: !Ref AvailabilityZone
      MapPublicIpOnLaunch: true
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-public-subnet'
        - Key: Tier
          Value: public

  PrivateSubnet:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VpcId
      CidrBlock: !Ref PrivateSubnetCidr
      AvailabilityZone: !Ref AvailabilityZone
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-private-subnet'
        - Key: Tier
          Value: private

Outputs:
  PublicSubnetId:
    Value: !Ref PublicSubnet
  PrivateSubnetId:
    Value: !Ref PrivateSubnet
```

Phân tích kỹ thuật: Module thực hiện phân mảnh không gian mạng dựa trên cờ MapPublicIpOnLaunch.

- Tại **PublicSubnet**, cờ này được thiết lập true để tự động cấp phát IP công cộng cho các máy chủ cần giao tiếp bên ngoài.
- Tại **PrivateSubnet**, tính năng này bị vô hiệu hóa, biến vùng mạng này thành một khu vực cách ly an toàn, tàng hình trước Internet.

→ Tham số AvailabilityZone được truyền trực tiếp từ Parent Stack, giúp kiểm soát và đồng bộ vị trí Data Center một cách chính xác.

2.3.3. Template NAT Gateway (nat.yaml)

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'NAT Gateway + Elastic IP'

Parameters:
  NamePrefix:
    Type: String
  PublicSubnetId:
    Type: AWS::EC2::Subnet::Id

Resources:
  NatEip:
    Type: AWS::EC2::EIP
    Properties:
      Domain: vpc
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-nat-eip'

  NatGateway:
    Type: AWS::EC2::NatGateway
    Properties:
      AllocationId: !GetAtt NatEip.AllocationId
      SubnetId: !Ref PublicSubnetId
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-nat-gateway'

Outputs:
  NatGatewayId:
    Value: !Ref NatGateway
```


Phân tích kỹ thuật: NAT Gateway yêu cầu một địa chỉ tĩnh Elastic IP để định tuyến giao thông mạng. Tự động tạo ra một liên kết phụ thuộc ngầm. CloudFormation sẽ block tiến trình tạo NAT Gateway cho đến khi EIP hoàn tất cấp phát thành công, giúp loại bỏ hoàn toàn lỗi khởi tạo tài nguyên khi cấu hình tiền đề chưa sẵn sàng.

2.3.4. Template Route Tables (route-tables.yaml)

```
Resources:
  PublicRouteTable:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref VpcId
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-public-rt'

  PublicRoute:
    Type: AWS::EC2::Route
    Properties:
      RouteTableId: !Ref PublicRouteTable
      DestinationCidrBlock: 0.0.0.0/0
      GatewayId: !Ref InternetGatewayId

  PublicSubnetAssoc:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties:
      RouteTableId: !Ref PublicRouteTable
      SubnetId: !Ref PublicSubnetId

  PrivateRouteTable:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref VpcId
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-private-rt'

  PrivateRoute:
    Type: AWS::EC2::Route
    Properties:
      RouteTableId: !Ref PrivateRouteTable
      DestinationCidrBlock: 0.0.0.0/0
      NatGatewayId: !Ref NatGatewayId

  PrivateSubnetAssoc:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties:
      RouteTableId: !Ref PrivateRouteTable
      SubnetId: !Ref PrivateSubnetId
```

Phân tích kỹ thuật: Để điều hướng traffic, module chia rẽ luồng mạng thành hai quy tắc định tuyến hoàn toàn độc lập.

- **PublicRoute** đẩy toàn bộ gói tin ra ngoài Internet (0.0.0.0/0) thông qua cổng GatewayId.
- Ngược lại, **PrivateRoute** ép buộc luồng dữ liệu từ vùng nội bộ phải đi qua NatGatewayId.

Thông qua tài nguyên AWS::EC2::SubnetRouteTableAssociation, các chính sách này được áp đặt nghiêm ngặt lên từng vùng mạng tương ứng, hình thành kiến trúc mạng an toàn nhiều lớp.

2.3.5. Template Security Groups (security-groups.yaml)

```
Resources:
  PublicSg:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: Allow SSH from specific IP
      VpcId: !Ref VpcId
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 22
          ToPort: 22
          CidrIp: !Ref MyIp
          Description: SSH from my IP
      SecurityGroupEgress:
        - IpProtocol: -1
          CidrIp: 0.0.0.0/0
          Description: Allow all outbound
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-public-sg'

  PrivateSg:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: Allow SSH only from Public SG
      VpcId: !Ref VpcId
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 22
          ToPort: 22
          SourceSecurityGroupId: !Ref PublicSg
          Description: SSH from Public EC2
      SecurityGroupEgress:
        - IpProtocol: -1
          CidrIp: 0.0.0.0/0
          Description: Allow all outbound
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-private-sg'
```

Phân tích kỹ thuật: Security Group được thiết lập dựa trên nguyên Least Privilege.

- Tại **PublicSg**, truy cập SSH Port 22 bị khóa cứng vào biến MyIp của người quản trị, đồng thời tích hợp cơ chế AllowedPattern Regex nhằm đảm bảo IP truyền vào luôn đúng định dạng.
- Tại **PrivateSg**, tính năng Cross-referencing được kích hoạt qua thuộc tính SourceSecurityGroupId: !Ref PublicSg.

→ Cơ chế này tạo ra một rào chắn an toàn: chỉ các máy ảo đang gắn giáp PublicSg mới có quyền kết nối vào máy chủ Private, tạo lập thành công mô hình Bastion Host.

2.3.6. Template EC2 (ec2.yaml)

```
Resources:
  PublicEc2:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !Ref AmiId
      InstanceType: !Ref InstanceType
      KeyName: !Ref KeyName
      SubnetId: !Ref PublicSubnetId
      SecurityGroupIds:
        - !Ref PublicSgId
      Monitoring: true
      MetadataOptions:
        HttpTokens: required
        HttpEndpoint: enabled
      BlockDeviceMappings:
        - DeviceName: /dev/xvda
          Ebs:
            Encrypted: true
            VolumeSize: 8
            VolumeType: gp3
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-public-ec2'
        - Key: Tier
          Value: public

  PrivateEc2:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !Ref AmiId
      InstanceType: !Ref InstanceType
      KeyName: !Ref KeyName
      SubnetId: !Ref PrivateSubnetId
      SecurityGroupIds:
        - !Ref PrivateSgId
      Monitoring: true
      MetadataOptions:
        HttpTokens: required
        HttpEndpoint: enabled
      BlockDeviceMappings:
        - DeviceName: /dev/xvda
          Ebs:
            Encrypted: true
            VolumeSize: 8
            VolumeType: gp3
      Tags:
        - Key: Name
          Value: !Sub '${NamePrefix}-private-ec2'
        - Key: Tier
          Value: private
```

Phân tích kỹ thuật: Module đóng vai trò cấp phát điện toán đám mây. Việc đảm bảo an toàn thiết bị đầu cuối được tăng cường qua hai cấu hình chuẩn Production:

- Thứ nhất, khối **MetadataOptions** buộc bật HttpTokens: required nhằm yêu cầu sử dụng giao thức IMDSv2, bảo vệ hệ thống trước các cuộc tấn công đánh cắp chứng chỉ SSRF.
- Thứ hai, khối **BlockDeviceMappings** cấu hình phân vùng lưu trữ sử dụng chuẩn ổ cứng gp3 hiệu năng cao kèm thuộc tính Encrypted: true để tự động mã hóa toàn phần dữ liệu lưu trữ.

2.3.7. Parent Stack (main.yaml)

```
Resources:
  VpcStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: !Sub 'https://${TemplateBucket}.s3.amazonaws.com/templates/vpc.yaml'
      Parameters:
        NamePrefix: !Ref NamePrefix
        VpcCidr: !Ref VpcCidr

  SubnetsStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: !Sub 'https://${TemplateBucket}.s3.amazonaws.com/templates/subnets.yaml'
      Parameters:
        NamePrefix: !Ref NamePrefix
        VpcId: !GetAtt VpcStack.Outputs.VpcId
        PublicSubnetCidr: !Ref PublicSubnetCidr
        PrivateSubnetCidr: !Ref PrivateSubnetCidr
        AvailabilityZone: !Ref AvailabilityZone

  NatStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: !Sub 'https://${TemplateBucket}.s3.amazonaws.com/templates/nat.yaml'
      Parameters:
        NamePrefix: !Ref NamePrefix
        PublicSubnetId: !GetAtt SubnetsStack.Outputs.PublicSubnetId

  RouteTablesStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: !Sub 'https://${TemplateBucket}.s3.amazonaws.com/templates/route-tables.yaml'
      Parameters:
        NamePrefix: !Ref NamePrefix
        VpcId: !GetAtt VpcStack.Outputs.VpcId
        InternetGatewayId: !GetAtt VpcStack.Outputs.InternetGatewayId
        NatGatewayId: !GetAtt NatStack.Outputs.NatGatewayId
        PublicSubnetId: !GetAtt SubnetsStack.Outputs.PublicSubnetId
        PrivateSubnetId: !GetAtt SubnetsStack.Outputs.PrivateSubnetId
```

```
SecurityGroupsStack:
  Type: AWS::CloudFormation::Stack
  Properties:
    TemplateURL: !Sub 'https://${TemplateBucket}.s3.amazonaws.com/templates/security-groups.yaml'
    Parameters:
      NamePrefix: !Ref NamePrefix
      VpcId: !GetAtt VpcStack.Outputs.VpcId
      MyIp: !Ref MyIp

Ec2Stack:
  Type: AWS::CloudFormation::Stack
  Properties:
    TemplateURL: !Sub 'https://${TemplateBucket}.s3.amazonaws.com/templates/ec2.yaml'
    Parameters:
      NamePrefix: !Ref NamePrefix
      AmiId: !Ref AmiId
      InstanceType: !Ref InstanceType
      KeyName: !Ref KeyName
      PublicSubnetId: !GetAtt SubnetsStack.Outputs.PublicSubnetId
      PrivateSubnetId: !GetAtt SubnetsStack.Outputs.PrivateSubnetId
      PublicSgId: !GetAtt SecurityGroupsStack.Outputs.PublicSgId
      PrivateSgId: !GetAtt SecurityGroupsStack.Outputs.PrivateSgId
```

Phân tích kỹ thuật: Thay vì khai báo tài nguyên vật lý trực tiếp, main.yaml sử dụng tài nguyên đặc biệt AWS::CloudFormation::Stack để gọi các module con đã được lưu trữ trên S3. Mấu chốt của kiến trúc Nested Stacks nằm ở luồng chuyển dữ liệu. Parent Stack trích xuất giá trị từ module trước (ví dụ: !GetAtt VpcStack.Outputs.VpcId) và truyền nó làm Parameter đầu vào cho module tiếp theo.

→ Quá trình trao đổi Output - Input này giúp CloudFormation tự động thiết lập cây phụ thuộc, đảm bảo hạ tầng được xây dựng theo đúng thứ tự logic mà không cần cấu hình chờ thủ công.

2.4. Quy trình triển khai

Quá trình triển khai hạ tầng được thực hiện tự động thông qua các câu lệnh cốt lõi AWS, thực hiện trực tiếp trên Terminal Windows – đã được cài đặt AWS CLI Extension thông qua câu lệnh:

```
msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

- Bước 1: Thực hiện tạo bucket và đẩy dữ liệu cho hạ tầng

```
aws s3 mb | aws s3 cp
```

Để bắt đầu xây dựng và khởi tạo môi trường, chạy lệnh tạo bucket hạ tầng, đồng thời upload toàn bộ source code .yaml lên Cloud trước khi tiến hành khởi tạo Parent Stack.

```
PS D:\UIT Storage\HK6 (2025-2026)\Công nghệ DevOps và Ứng dụng\devops-aws-microservices\lab01\cloudformation> aws s3 mb s3://nt548-cfn-templates-nhom03 --region ap-southeast-1
make_bucket: nt548-cfn-templates-nhom03
PS D:\UIT Storage\HK6 (2025-2026)\Công nghệ DevOps và Ứng dụng\devops-aws-microservices\lab01\cloudformation> aws s3 cp templates/ s3://nt548-cfn-templates-nhom03/templates/ --recursive
upload: templates\security-groups.yaml to s3://nt548-cfn-templates-nhom03/templates/security-groups.yaml
upload: templates\route-tables.yaml to s3://nt548-cfn-templates-nhom03/templates/route-tables.yaml
upload: templates\vpc.yaml to s3://nt548-cfn-templates-nhom03/templates/vpc.yaml
upload: templates\ec2.yaml to s3://nt548-cfn-templates-nhom03/templates/ec2.yaml
upload: templates\subnets.yaml to s3://nt548-cfn-templates-nhom03/templates/subnets.yaml
upload: templates\nat.yaml to s3://nt548-cfn-templates-nhom03/templates/nat.yaml
```

- Bước 2: Thực hiện xây dựng Stack tự động hóa lên Cloudformation

```
aws cloudformation deploy
```

Sử dụng toàn bộ cấu hình từ các file templates, stack name, parameter, ... vừa được upload lên Cloud để hệ thống tự động hóa việc đọc – rà soát dữ liệu – cài đặt nền tảng Stack lên Cloudformation.

```
PS D:\UIT Storage\HK6 (2025-2026)\Công nghệ DevOps và Ứng dụng\devops-aws-microservices\lab01\cloudformation> aws cloudformation delete-stack --stack-name nt548-lab01-nhom03
PS D:\UIT Storage\HK6 (2025-2026)\Công nghệ DevOps và Ứng dụng\devops-aws-microservices\lab01\cloudformation> aws cloudformation deploy --template-file main.yaml --stack-name nt548-lab01-nhom03 --parameter-overrides file://parameters.json --capabilities CAPABILITY_NAMED_IAM --region ap-southeast-1

Waiting for changeset to be created..
Waiting for stack create/update to complete
Successfully created/updated stack - nt548-lab01-nhom03
```

→ Thông báo triển khai thành công Stack lên Cloudformation.

- Bước 3: Trích xuất dữ liệu lõi từ hạ tầng

```
aws cloudformation describe-stack
```

Sử dụng tính năng Describe Stack từ Cloudformation để truy vấn dữ liệu ở VPC, EC2 IP, ta thu được một bảng tóm gọn như hình, cụ thể:

- VPC ID: vpc-05c52afed95e1a0b1
- EC2 Public IP: 13.228.30.8
- EC2 Private IP: 10.0.2.103

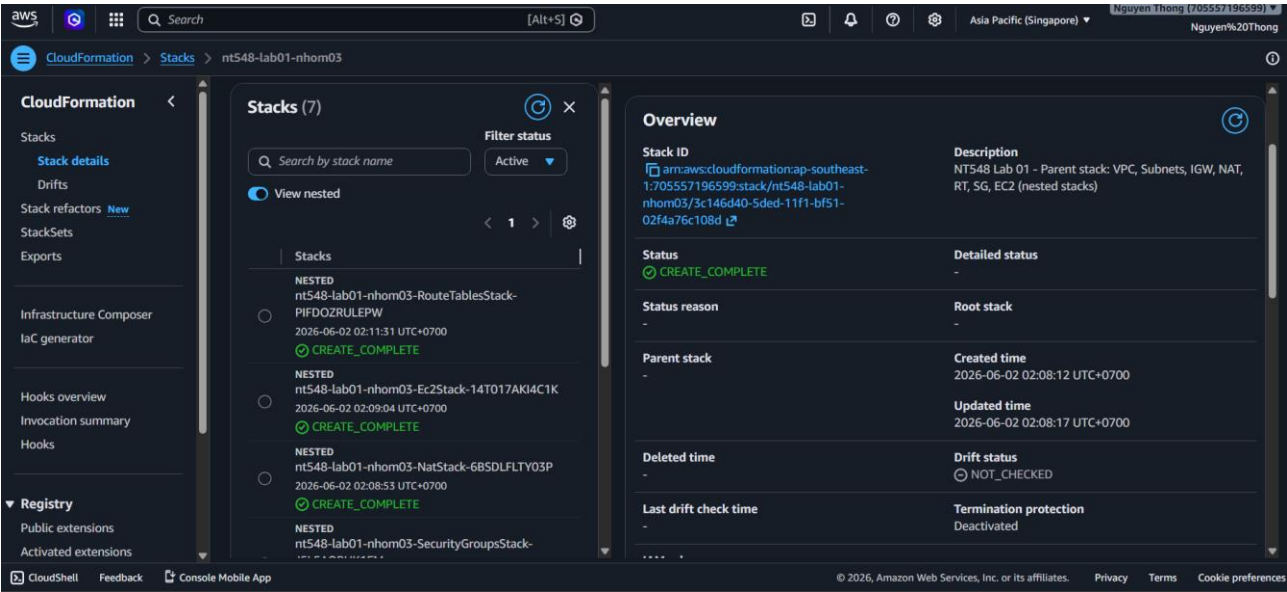
```
PS D:\UIT Storage\HK6 (2025-2026)\Công nghệ DevOps và Ứng dụng\devops-aws-microservices\lab01\cloudformation> aws cloudformation describe-stacks --stack-name nt548-lab01-nhom03 --query 'Stacks[0].Outputs' --output table
>>
+-----+-----+
| DescribeStacks |
+-----+-----+
| OutputKey | OutputValue |
+-----+-----+
| VpcId | vpc-05c52afed95e1a0b1 |
| PrivateEc2PrivateIp | 10.0.2.103 |
| PublicEc2PublicIp | 13.228.30.8 |
+-----+-----+
```

2.5. Xác minh kết quả trên giao diện AWS (AWS Console)

Sau khi quá trình tự động hóa hoàn tất, tiến hành kiểm tra trên giao diện của AWS Console để xác nhận hạ tầng thực tế đã khớp với mã nguồn.

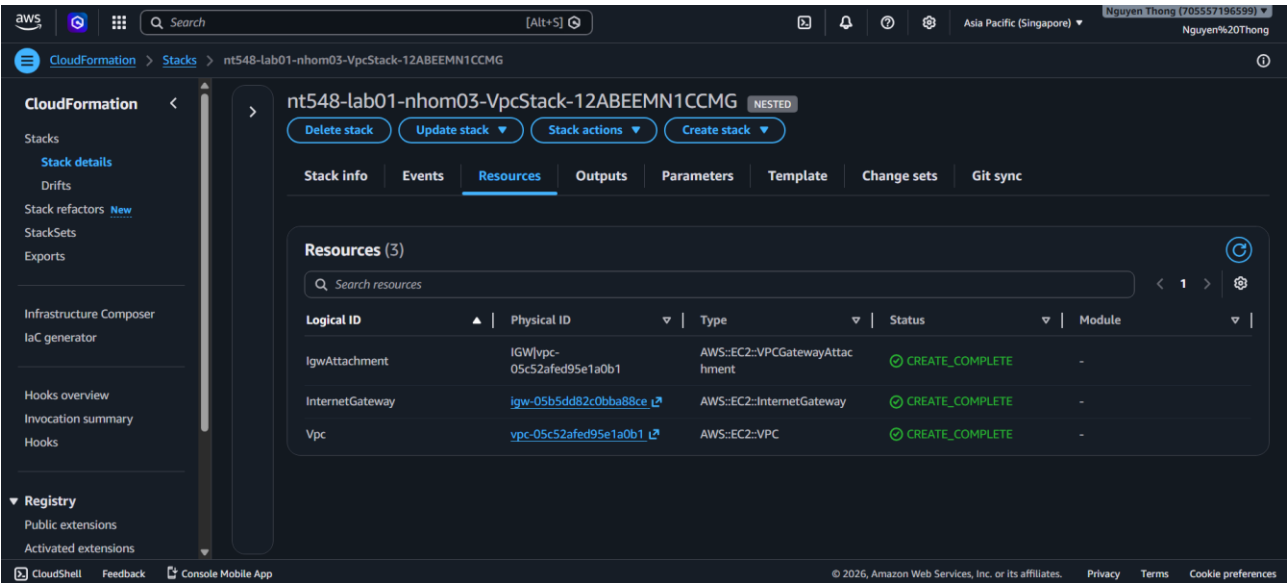
- **Kiểm tra Nested Stack:** Tại trang Stack Details của Cloudformation, kiểm tra Stack hạ tầng chính, gồm tổng cộng 7 stacks với nhiều dịch vụ được triển khai.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS



➔ **Phân tích:** Xây dựng thành công Nested Stack, bao hàm các dịch vụ triển khai như EC2, VPC, Route Table, NAT Gateway, ...

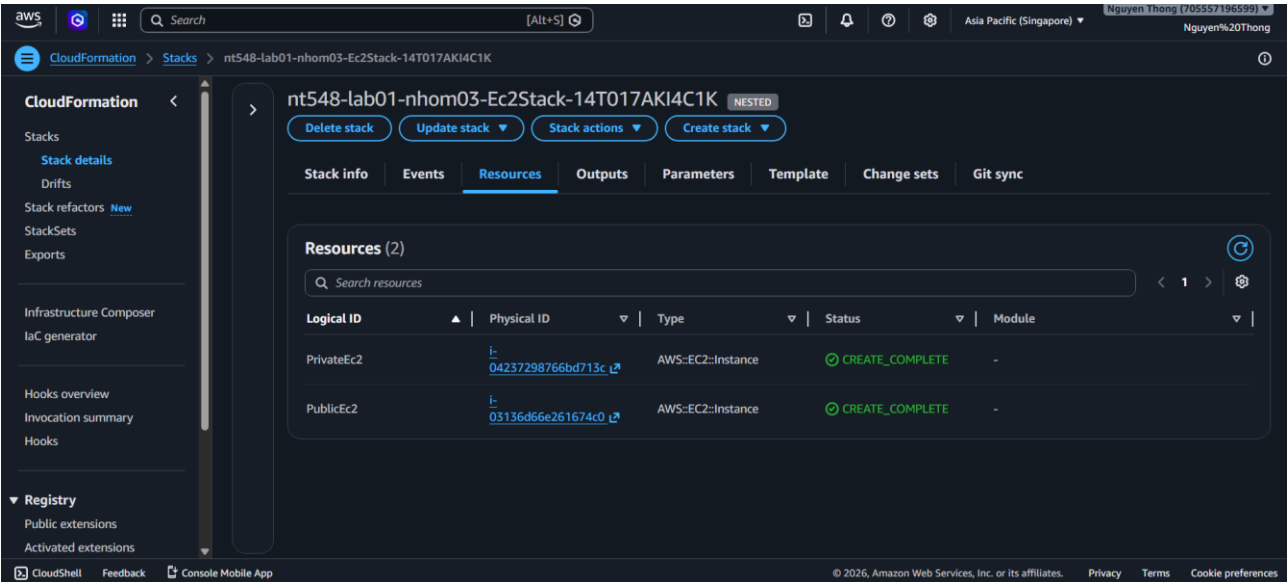
- **Kiểm tra VPC Stack:** Truy cập VpcStack được Nest bên trong Stack hạ tầng chính.



➔ **Phân tích:** VPC được triển khai thành công với chính xác VPC ID được trích xuất từ output Describe Stack ở bước deploy phía trên, được gắn nhãn CREATE_COMPLETE, đồng thời được attach đúng các gateway.

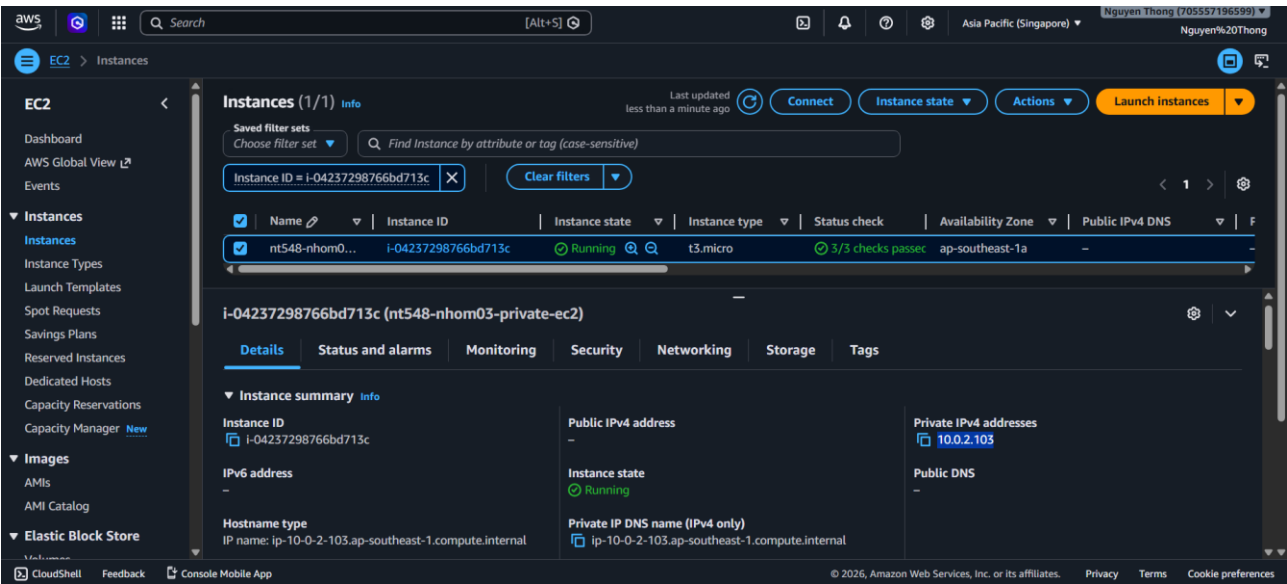
- **Kiểm tra EC2 Stack:** Truy cập EC2Stack được Nest bên trong Stack hạ tầng chính.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS



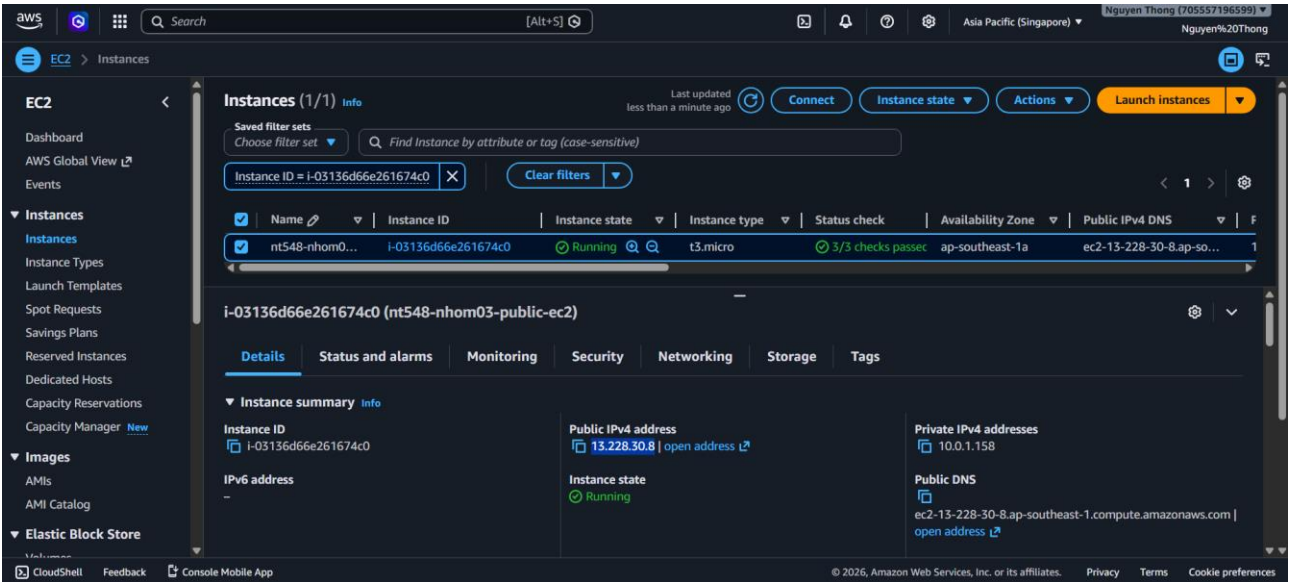
➔ Phân tích: EC2 được triển khai thành công với cụ thể Public IP và Private IP tương ứng:

- EC2 Private IP

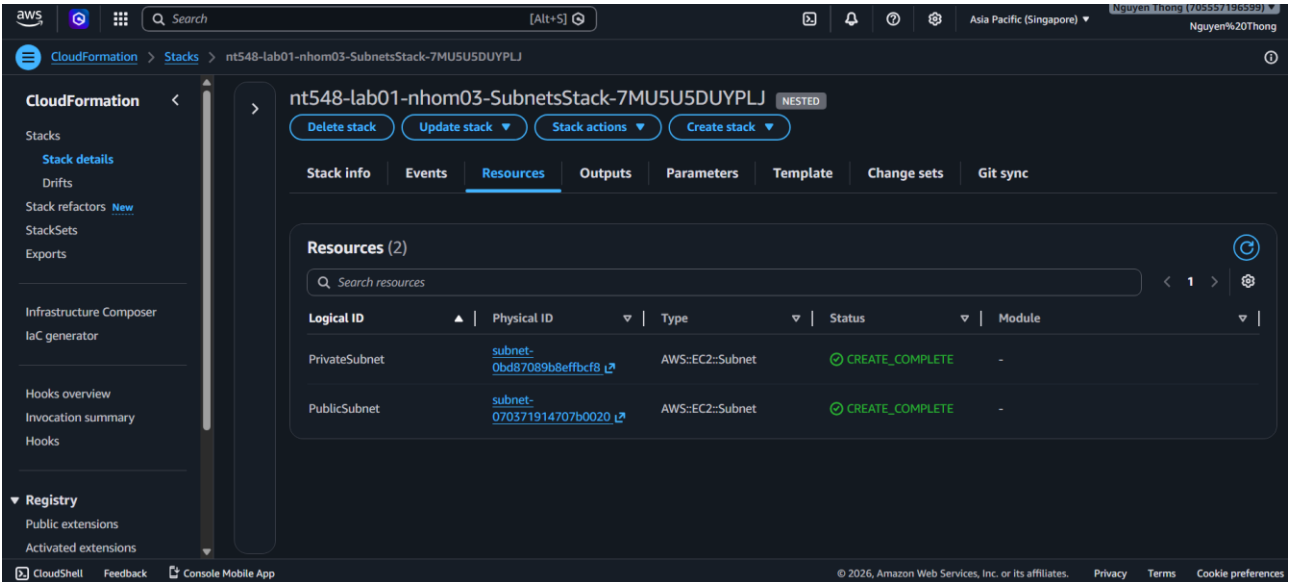


- EC2 Public IP

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

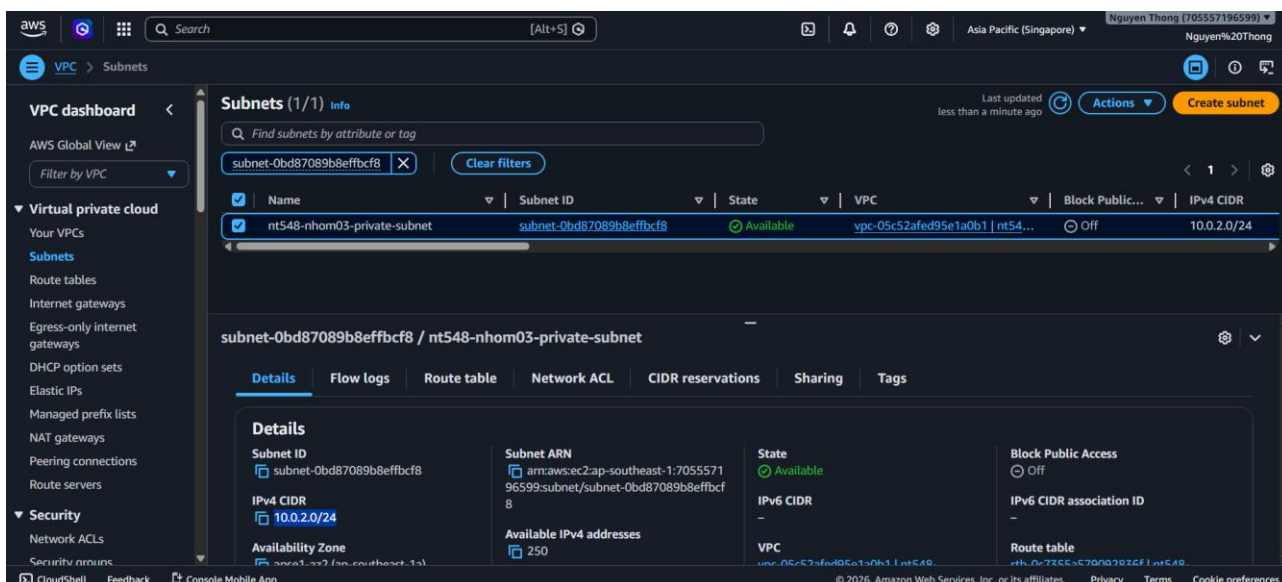


- Kiểm tra Subnet Stack: Truy cập SubnetStack được Nest bên trong Stack hạ tầng chính.

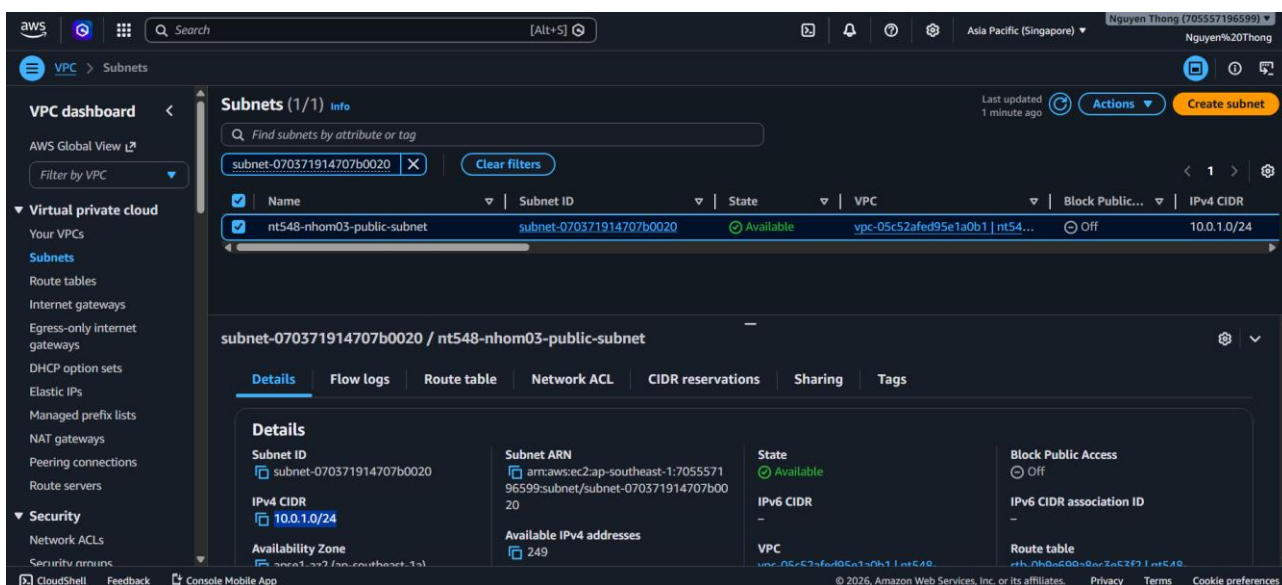


- ➔ Phân tích: Subnet được triển khai thành công với cụ thể Public Subnet và Private Subnet tương ứng cấu hình resource CIDR ở file templates:
 - Private Subnet

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

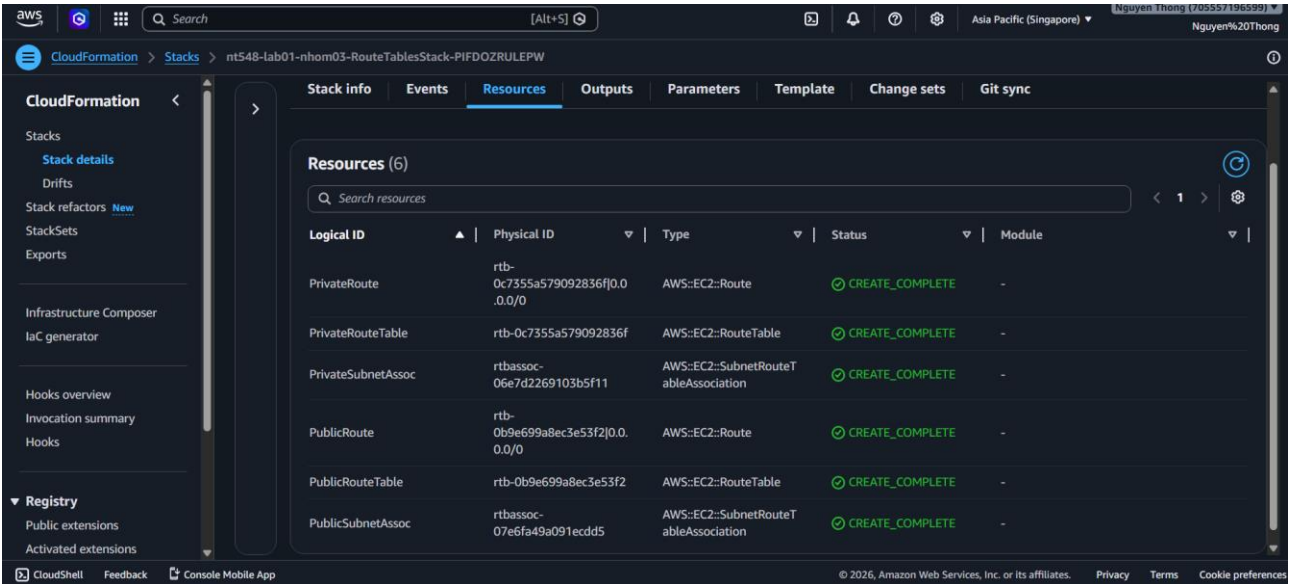


○ Public Subnet

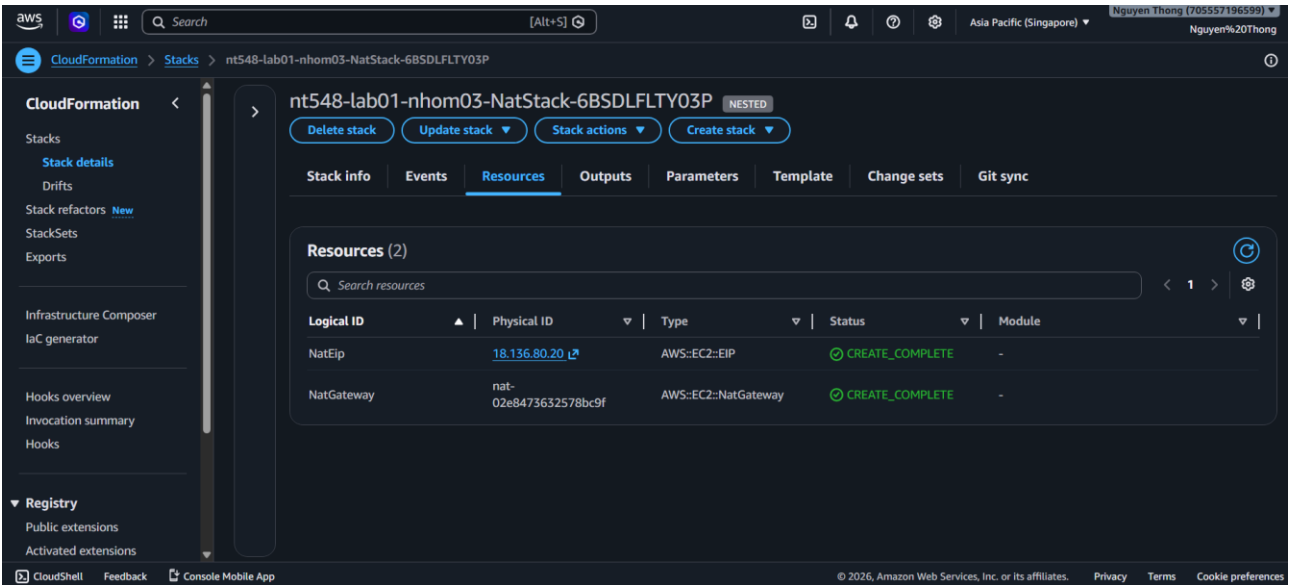


- **Kiểm tra Route Table Stack:** Truy cập RouteTablesStack được Nest bên trong Stack hạ tầng chính.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

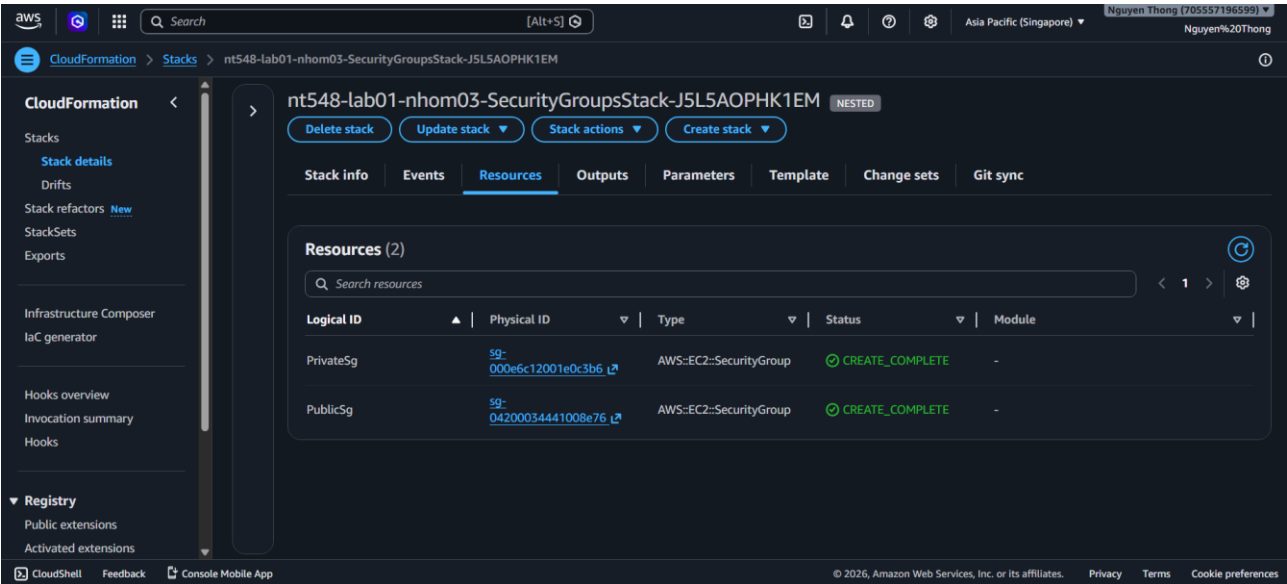


- ➔ **Phân tích:** Route Tables được cài đặt thành công tương ứng lần lượt với các Private / Public Route trong template.
- **Kiểm tra NAT Stack:** Truy cập NATStack được Nest bên trong Stack hạ tầng chính.



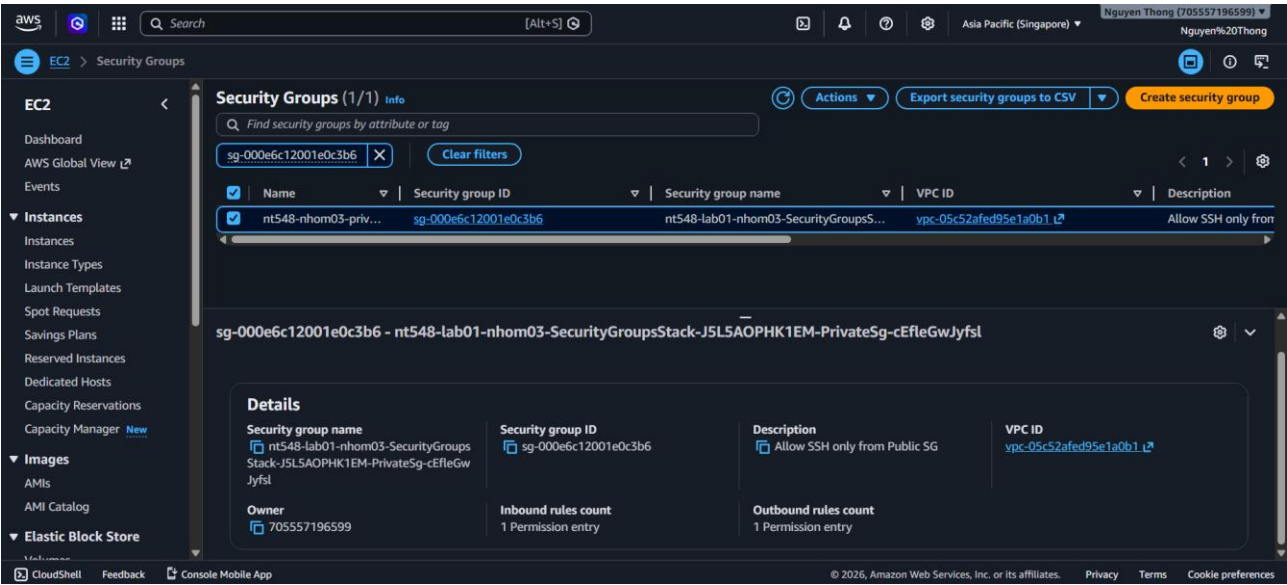
- ➔ **Phân tích:** NAT Stack được cài đặt thành công tương ứng lần lượt với NatEip và NATGateway trong template.
- **Kiểm tra Security Stack:** Truy cập SecurityGroupsStack được Nest bên trong Stack hạ tầng chính.

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS



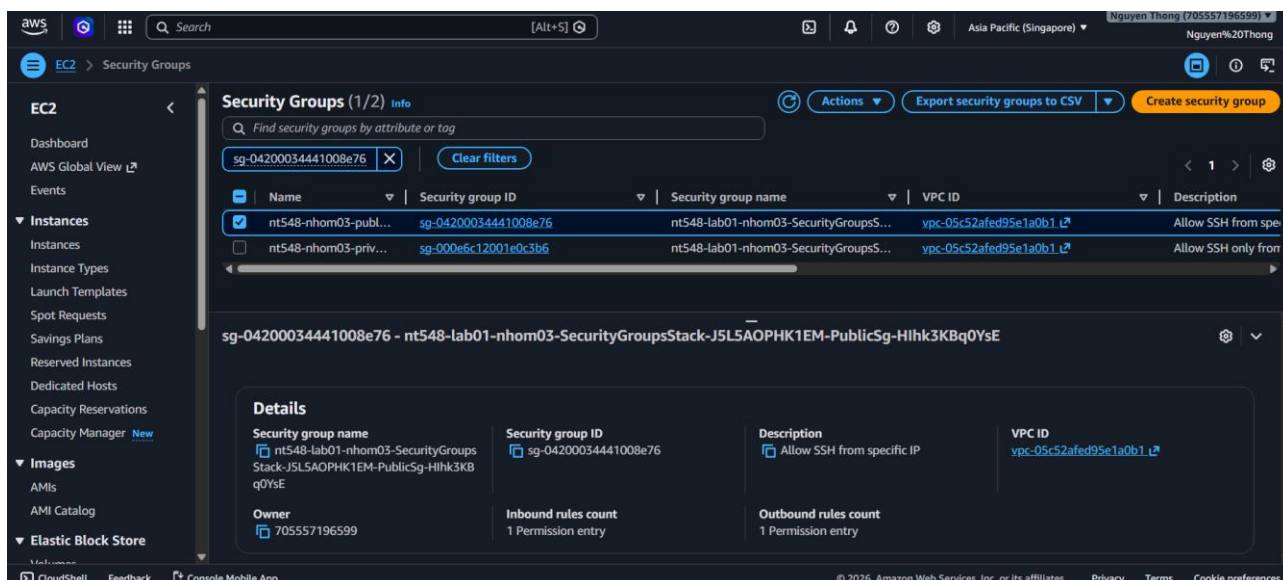
➔ Phân tích: Security Groups được cài đặt tuân theo đúng mong muốn cho PrivateSG / PublicSG..

○ Private Security Group



○ Public Security Group

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS



➔ **Tóm lại:** Hạ tầng đã được tự động hóa triển khai theo đúng yêu cầu cấu hình, sử dụng đầy đủ resource từ templates.

2.6. Test cases và xác minh

Để đảm bảo các dịch vụ được triển khai thành công ở bước hậu cấu hình, ta thực hiện các kịch bản kiểm thử đối với từng dịch vụ thông qua file scripts Powershell – **run-tests.ps1**.

Lệnh thực thi Script tự động hóa kiểm thử kịch bản:

```
.\run-tests.ps1 -KeyPath "D:\aws_key\nt548-keypair.pem" -PublicIP "13.228.30.8" -  
PrivateIP "10.0.2.103"
```

Với các tham số:

- **KeyPath:** nt548-keypair.pem
- **PublicIP:** 13.228.30.8
- **PrivateIP:** 10.0.2.103

GIẢI ĐOẠN 1: KIỂM TRA TÍNH SẴN SÀNG CỦA TÀI NGUYÊN

- **Mục đích:** Xác nhận các tài nguyên mạng cốt lõi (VPC, Subnets, NAT Gateway, IGW) tồn tại và ở trạng thái available.
- **Cơ chế chung:** Dùng AWS CLI filter theo Tag Name được định nghĩa trong file parameter, nếu tài nguyên tồn tại và báo available → PASS.

- Test 1 – VPC Check

```
$vpc = aws ec2 describe-vpcs --filters "Name=tag:Name,Values=$Prefix-vpc" --region $Region --query "Vpcs[0].State" --output text  
Write-Result "1. Kiểm tra mạng ao VPC ($Prefix-vpc)" ($vpc -eq "available") "State: $vpc"
```

- Test 2- Subnet Check

```
$pubSub = aws ec2 describe-subnets --filters "Name=tag:Name,Values=$Prefix-public-subnet" --region $Region --query "Subnets[0].State" --output text  
Write-Result "2. Kiểm tra Public Subnet" ($pubSub -eq "available") "State: $pubSub"  
  
$privSub = aws ec2 describe-subnets --filters "Name=tag:Name,Values=$Prefix-private-subnet" --region $Region --query "Subnets[0].State" --output text  
Write-Result "3. Kiểm tra Private Subnet" ($privSub -eq "available") "State: $privSub"
```


Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

- Test 3 – Gateway Check

```
$igw = aws ec2 describe-internet-gateways --filters "Name=tag:Name,Values=$Prefix-igw" --region $Region --query "InternetGateways[0].Attachments[0].State"
Write-Result "4. Kiểm tra Internet Gateway" ($igw -eq "available") "Attachment State: $igw"

$nat = aws ec2 describe-nat-gateways --filter "Name=tag:Name,Values=$Prefix-nat-gateway" --region $Region --query "NatGateways[0].State" --output text
Write-Result "5. Kiểm tra NAT Gateway" ($nat -eq "available") "State: $nat"
```

- Test 4 – EC2 Instances Check

```
$pubEc2 = aws ec2 describe-instances --filters "Name=tag:Name,Values=$Prefix-public-ec2" --region $Region --query "Reservations[0].Instances[0].State.Name"
Write-Result "6. Kiểm tra Public EC2 Instance" ($pubEc2 -eq "running") "State: $pubEc2"

$privEc2 = aws ec2 describe-instances --filters "Name=tag:Name,Values=$Prefix-private-ec2" --region $Region --query "Reservations[0].Instances[0].State.Name"
Write-Result "7. Kiểm tra Private EC2 Instance" ($privEc2 -eq "running") "State: $privEc2"
```

===== GIAI DOAN 1: KIEM TRA TRANG THAI TAI NGUYEN =====

```
1. Kiểm tra mạng ao VPC (nt548-nhom03-vpc) [PASS]
   -> State: available
2. Kiểm tra Public Subnet [PASS]
   -> State: available
3. Kiểm tra Private Subnet [PASS]
   -> State: available
4. Kiểm tra Internet Gateway [PASS]
   -> Attachment State: available
5. Kiểm tra NAT Gateway [PASS]
   -> State: available
6. Kiểm tra Public EC2 Instance [PASS]
   -> State: running
7. Kiểm tra Private EC2 Instance [PASS]
   -> State: running
```

- ➔ Sử dụng tính năng **Describe Stack Cloudformation**, kết quả thu được là kiểm nhận đạt – available dành cho các dịch vụ, cho thấy các dịch vụ này đang hoạt động bình thường, trơn tru, sẵn sàng cho việc kiểm tra kết nối.

GIAI ĐOẠN 2: KIỂM TRA KHẢ NĂNG GIAO TIẾP Ở CÁC DỊCH VỤ

- **Mục đích:** Xác nhận các kết nối, traffic giữa các dịch vụ được triển khai được trơn tru, thực sự hoạt động. Mỗi kiểm tra có cơ chế riêng biệt.

- Test 5 – Public EC2 Reachability & Internet Access Check

```
$sshPub = ssh -n -i $KeyPath -o StrictHostKeyChecking=no -o ConnectTimeout=5 ec2-user@$PublicIP "echo OK" 2>&1
Write-Result "8. Truy cập SSH vào Public EC2" ([bool]($sshPub -match "OK")) "Truy cập thông suốt"

$inetPub = ssh -n -i $KeyPath -o StrictHostKeyChecking=no -o ConnectTimeout=5 ec2-user@$PublicIP "curl -s -I https://aws.amazon.com | head -n 1" 2>&1
Write-Result "9. Public EC2 kết nối ra Internet" ([bool]($inetPub -match "HTTP")) "Truy cập Internet OK"
```

- **Cơ chế:** SSH trực tiếp từ máy local vào Public IP. Kết nối thành công sẽ cho thấy luồng Internet → IGW → Public Subnet → EC2 thông suốt. Từ bên trong

Bài TH số 01: Dùng Terraform và CloudFormation để quản lý và triển khai hạ tầng AWS

Public EC2, gọi lệnh curl *https://aws.amazon.com*, nếu trả về mã HTTP hợp lệ
→ PASS.

- Test 6 – Private EC2 Reachability & Internet Access Check

```
$sshPriv = ssh -n -i $KeyPath -o StrictHostKeyChecking=no ec2-user@$PublicIP "ssh -n -i mykey.pem -o StrictHostKeyChecking=no -o ConnectTimeout=5 ec2-user@$PrivateIP  
Write-Result "10. Truy cap SSH vao Private EC2 qua Bastion" ([bool]($sshPriv -match "OK")) "Jump thanh cong qua Bastion"  
  
$inetPriv = ssh -n -i $KeyPath -o StrictHostKeyChecking=no ec2-user@$PublicIP "ssh -n -i mykey.pem -o StrictHostKeyChecking=no -o ConnectTimeout=5 ec2-user@$PrivateIP  
Write-Result "11. Private EC2 ra Internet (qua NAT Gateway)" ([bool]($inetPriv -match "HTTP")) "NAT Gateway hoạt động hoàn hảo"
```

- **Cơ chế:** Sử dụng cơ chế SSH Jump, nhảy từ máy local xuyên qua Public EC2 để kết nối vào Private EC2 qua Private IP. Điều khiển Private EC2 qua SSH jump → chạy curl ra Internet. NAT Gateway dịch IP nội bộ 10.0.2.x sang Elastic IP để lấy dữ liệu về, tải thành công → PASS.

- Test 7 – Private EC2 Isolation Check

```
Write-Host "12. Kiem tra cach ly Private EC2 tu Internet " -NoNewline  
$isoTest = ssh -n -i $KeyPath -o StrictHostKeyChecking=no -o ConnectTimeout=3 ec2-user@$PrivateIP "echo BAD" 2>&1  
if (-not ($isoTest -match "BAD")) {  
    Write-Host "[PASS]" -ForegroundColor Green  
    Write-Host " -> Ket noi bi tu choi (Dung thiet ke bao mat)" -ForegroundColor DarkGray  
} else {  
    Write-Host "[FAIL]" -ForegroundColor Red  
    Write-Host " -> Lo hong: Co the truy cap truc tiep vao Private!" -ForegroundColor Red  
}
```

- **Cơ chế:** Cố tình SSH thẳng từ máy local vào Private IP để kết nối bắt buộc phải timeout/thất bại thì mới PASS, chứng minh kiến trúc bảo mật không có lỗ hổng.

```
=====
GIAI DOAN 2: KIEM THU LUONG MANG
=====

8. Truy cap SSH vao Public EC2 [PASS]
   -> Truy cap thong suot
9. Public EC2 ket noi ra Internet [PASS]
   -> Truy cap Internet OK
10. Truy cap SSH vao Private EC2 qua Bastion [PASS]
    -> Jump thanh cong qua Bastion
11. Private EC2 ra Internet (qua NAT Gateway) [PASS]
    -> NAT Gateway hoạt động hoàn hảo
12. Kiem tra cach ly Private EC2 tu Internet [PASS]
    -> Ket noi bi tu choi (Dung thiet ke bao mat)
```

- ➔ **Kết luận:** Toàn bộ các tài nguyên vật lý và logic (VPC, Subnets, Gateways, EC2) đã được AWS CloudFormation biên dịch và cấp phát đầy đủ, chính xác theo kiến trúc Nested Stacks. Luồng dữ liệu hoạt động trơn tru đúng theo thiết kế. Phân vùng Public giao tiếp trực tiếp qua Internet Gateway, trong khi phân vùng Private định tuyến gián tiếp qua NAT Gateway để kết nối ra bên ngoài một cách ổn định.

Câu 3 — Tổng kết và so sánh

3.1. So sánh Terraform và CloudFormation

Tiêu chí	Terraform	CloudFormation
Ngôn ngữ	HCL – ngắn gọn, hỗ trợ biểu thức, hàm tích hợp	YAML/JSON – quen thuộc, verbose hơn
Đa cloud	AWS, Azure, GCP, Kubernetes,... (cloud-agnostic)	Chỉ AWS
State	Tự quản lý (S3 + DynamoDB lock khi làm nhóm)	AWS tự quản lý – không lo state file
Module/Tái sử dụng	Module linh hoạt, publish lên Terraform Registry	Nested Stacks (phụ thuộc S3 bucket cùng region)
Rollback lỗi	Dùng giữa chừng, rollback thủ công	Tự động rollback toàn stack khi lỗi
Drift Detection	Thủ công: terraform plan	Có tích hợp sẵn trong Console
CI/CD	GitHub Actions, Atlantis, Terraform Cloud (linh hoạt)	Native với CodePipeline, CodeBuild
Multi-account	Cần cấu hình thêm	StackSets – deploy nhiều account 1 lần

➔ Kinh nghiệm: Khi nào chọn?

- **Terraform** khi: hạ tầng multi-cloud, team DevOps mạnh, cần tái sử dụng module rộng rãi, CI/CD linh hoạt ngoài AWS ecosystem.
- **CloudFormation** khi: toàn bộ stack nằm trong AWS, muốn rollback tự động, deploy multi-account qua StackSets, không muốn quản lý state file.

3.2. Bài học kinh nghiệm

Vấn đề	Bài học / Giải pháp
IAM Permissions	Cần đủ quyền cho tất cả resource trước khi apply. Dùng Least Privilege ở production.
State file (Terraform)	Cấu hình S3 backend + DynamoDB lock ngay từ đầu khi làm nhóm, tránh xung đột.
Race Condition – NAT GW	Dùng depends_on = [aws_eip.nat] để Terraform chờ EIP xong mới tạo NAT Gateway.
Nested Stack – Output/Input	Mỗi child stack phải khai báo Outputs đầy đủ; parent dùng !GetAtt để nối dữ liệu.

S3 cho CFN Templates	Upload templates lên S3 trước, bucket phải cùng Region với stack.
SG Cross-referencing	Dùng ID Security Group làm source thay vì IP – chuẩn pattern Bastion Host.
terraform fmt & validate	Chạy trước apply để phát hiện lỗi sớm, không cần chờ AWS báo lỗi.

3.3. Kết luận

Kết quả đạt được

- Triển khai thành công cùng một kiến trúc AWS bằng cả Terraform và CloudFormation, bao gồm đầy đủ: VPC, Public/Private Subnet, IGW, NAT Gateway, Route Tables, Security Groups (Least Privilege + Bastion Host pattern), EC2 Instances.
- Toàn bộ được tổ chức dưới dạng module/nested stack, tái sử dụng được.
- Các test cases tự động vượt qua.

Nhận xét

- Terraform mạnh về tính linh hoạt, đa cloud và hệ sinh thái module. Phù hợp khi cần quản lý hạ tầng nhiều nền tảng cùng lúc.
- CloudFormation mạnh về tích hợp native AWS, rollback tự động và zero state management. Phù hợp cho môi trường production thuần AWS.
- IaC không chỉ là tự động hóa – đây là cách biến hạ tầng thành source of truth: version control, review, rollback như code thông thường.
- Kỹ năng thực tế quan trọng nhất: hiểu dependency giữa các resource, quản lý state đúng cách, và thiết kế security từ đầu chứ không phải vá sau.

Link GitHub: <https://github.com/DoanThao013/devops-aws-microservices>

PHỤ LỤC

A. Hướng dẫn chạy mã nguồn

Xem README.md trong repo GitHub.

B. Tham khảo

- Terraform AWS Provider Documentation:
<https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- AWS CloudFormation User Guide:
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html>
- AWS VPC Documentation: <https://docs.aws.amazon.com/vpc/>

HẾT